

DynHomotopy: Reproducing, Auditing and Improving a Dynamic Homotopy Method for Flat-Start Power Flow on Ill-Conditioned Grids

CSE 402 Numerical Analysis Sessional Project Report • Section A • Group 12

Mohiuddin Sizan
Student ID: 2105039
Dept. of CSE, BUET
Dhaka, Bangladesh

Sadman bin Tareq
Student ID: 2105040
Dept. of CSE, BUET
Dhaka, Bangladesh

Mosharaf Hossain Apurbo
Student ID: 2105057
Dept. of CSE, BUET
Dhaka, Bangladesh

Elias Hasbi
Student ID: 2105058
Dept. of CSE, BUET
Dhaka, Bangladesh

Jonayed Mohiuddin Shanto
Student ID: 2105060
Dept. of CSE, BUET
Dhaka, Bangladesh

Abstract

Newton–Raphson (NR), the standard power flow solver, diverges from the usual flat start on large, ill-conditioned grids. Lima-Silva and Freitas (2024) proposed embedding the power flow equations in the homotopy $G(x, t) = t g(x) + (1 - t)K(x - x^{(0)})$, integrating the resulting ordinary differential equation with two or three coarse backward-Euler (BE) steps, and refining the result with NR or the fast decoupled method. We implemented the method in Python, reproduced every table and figure of the paper, and widened the evaluation from nine to 34 public systems of up to 109,272 buses, counting cost in LU factorizations and a run as solved only if it reaches the reference operating point. Of the 255 mismatch norms in the paper’s Tables 2–5, 246 agree with ours within a factor of two, and 54 of its 63 iteration counts in Table 7 agree exactly; the rest trace to eight inconsistencies in the paper. We show that the recommended two-point BE path is a Jacobian-shifted Newton step followed by an almost plain Newton step. On the wider test bed the paper’s method converges in 141 of 170 runs but reaches the reference solution in only 127. A Newton corrector after each path step raises this to 152 without losing any run, and together with Iwamoto’s optimal multiplier it keeps that robustness at 7% fewer factorizations. Richardson step control solves systems nothing else solves, at 18–30% more cost. Spectral, feasibility and golden-section tuning studies show that the shift $\delta = K/\Delta t_0$ matters only on ill-conditioned grids and no simple rule predicts its best value.

CCS Concepts

• **Mathematics of computing** → Ordinary differential equations; Solvers; Computations on matrices.

Keywords

power flow, Newton–Raphson, dynamic homotopy, backward Euler, fast decoupled load flow, ill-conditioned systems, reproducibility, Richardson extrapolation, sparse LU

1 Introduction and Base Paper Review

1.1 Background: the power flow problem

Whenever a grid operator asks how the voltages across a network will settle for a given pattern of generation and demand—before switching a line out, after a generator trips, while planning next year’s load—a *power flow* problem is solved. For every bus the power injected by generators minus the power drawn by loads has to equal the power flowing out through the lines, which gives two nonlinear equations per bus in the unknown voltage magnitudes V and angles θ :

$$g(x) = 0, \quad x = [\theta_{PV}, \theta_{PQ}, V_{PQ}], \quad g: \mathbb{R}^n \rightarrow \mathbb{R}^n, \quad (1)$$

where n is 22 for the smallest system in this report and 185,807 for the largest.

With $Y = G + jB$ the bus admittance matrix and $V_k \angle \theta_k$ the voltage at bus k , the active and reactive mismatches are

$$\Delta P_k = V_k \sum_m V_m (G_{km} \cos \theta_{km} + B_{km} \sin \theta_{km}) - P_k^{\text{sp}}, \quad (2)$$

$$\Delta Q_k = V_k \sum_m V_m (G_{km} \sin \theta_{km} - B_{km} \cos \theta_{km}) - Q_k^{\text{sp}}, \quad (3)$$

with $\theta_{km} = \theta_k - \theta_m$. At the single *slack* bus V and θ are fixed; at *PV* (generator) buses P and V are specified; at *PQ* (load) buses both V and θ are unknown. Hence $n = N_{PV} + 2N_{PQ}$ and $g(x) = [\Delta P_{PV}, \Delta P_{PQ}, \Delta Q_{PQ}]$. In complex form the whole mismatch is $\Delta S = V \circ (\bar{Y}V) - S^{\text{sp}}$, with MATPOWER’s ordering and sign [29].

Newton–Raphson and fast decoupled load flow. NR updates $x^{(i+1)} = x^{(i)} - J(x^{(i)})^{-1}g(x^{(i)})$, where $J = \partial g / \partial x$ is sparse with the pattern of the network [23]; each iteration costs one sparse LU factorization. Near a solution it converges quadratically, but far from one it has no guarantee, and in practice it is usually started far away, from the *flat start* ($V = 1$ pu, $\theta = 0$ everywhere) because nothing better is known. The fast decoupled method [22] approximates J by two constant matrices, $\Delta P/V = B' \Delta \theta$ and $\Delta Q/V = B'' \Delta V$, factorised once and reused. In the XB variant [26] (FDXB) resistances are ignored only when building B' .

Homotopy methods. A homotopy replaces $g(x) = 0$ by a family $G(x, t) = 0$, $t \in [0, 1]$, whose member at $t = 0$ is trivial and whose member at $t = 1$ is the real problem [1, 28]. A *static* homotopy solves $G(x, t_k) = 0$ accurately with NR at each of several points t_k ; the GSH-NR method of [4], which adds fictitious shunts that make the flat start exact at $t = 0$, is of this kind, and every path point costs a full NR solve. A *dynamic* homotopy [16] instead differentiates $G(x(t), t) = 0$,

$$G_x \frac{dx}{dt} + G_t = 0 \implies \frac{dx}{dt} = -G_x(x, t)^{-1} G_t(x, t), \quad x(0) = x^{(0)}, \quad (4)$$

and *integrates* this ODE numerically, accepting that the result at $t = 1$ is only approximate. Continuous Newton methods [20] are a close relative.

1.2 Review of the base paper

Our base paper is A. Lima-Silva and F.D. Freitas, “Exploring a Dynamic Homotopy Technique to Enhance the Convergence of Classical Power Flow Iterative Solvers in Ill-Conditioned Power System Models,” *Energies* 17(18):4642, 2024 [17]. It uses the fixed-point homotopy

$$G(x, t) = t g(x) + (1 - t) K (x - x^{(0)}), \quad (5)$$

with $G_x = t J(x) + (1 - t) K I$ and $G_t = g(x) - K(x - x^{(0)})$. The factor K , which earlier work set to one, is the main design lever. The paper integrates (4) with forward Euler (FE), second-order Runge–Kutta (RK2) and a backward Euler (BE) scheme simplified to a single fixed-point iteration. It always takes the first step to $t_1 = \Delta t_0$ with BE and refines the point reached at $t = 1$ with NR or FDXB. It is tested on five ill-conditioned grids built by replicating PEGASE networks [2] (18,482 to 109,272 buses) and four loading-limit cases [24, 25], and reports that:

- FE and RK2 with large steps blow up on the big systems, while BE succeeds everywhere (Tables 2–4);
- BE with only two path points, $\{0, 0.005, 1\}$, lets NR converge in four iterations where NR from the flat start fails (Table 7);
- BE followed by FDXB is faster than NR from the stored operating point, at 57–85% of its CPU time (Table 8).

The paper’s future work names other homotopy functions, other integration methods and larger networks. Its strengths are a very cheap remedy (one LU per path point) and large realistic test systems. Its weaknesses, which we document in Section 4.2, are that the static baseline is described only in words, several reported numbers are mutually inconsistent, the choice of BE is justified mainly by experiment, and it does not report *which* of the several roots of $g(x) = 0$ a converged run reaches.

Related ideas we build on. Iwamoto and Tamura’s *optimal multiplier* [10] scales each NR step to minimise the mismatch along it. Predictor–corrector continuation [1] alternates a cheap predictor step with a Newton corrector back onto the path. Adding a multiple of the identity to an ill-conditioned Jacobian is the idea behind Levenberg–Marquardt damping [14, 19] and the conditioning step of [3]. The stability of implicit Euler on stiff problems is textbook material [7].

1.3 Objectives

This project set out to answer five questions:

- Q1. Does it reproduce?** Built from the paper’s equations alone, do we get its numbers?
- Q2. Why does it work?** Is there an algebraic reason why BE beats FE and RK2?
- Q3. Is the paper consistent?** Where we disagree, is the fault ours or the paper’s?
- Q4. Can it be improved, and at what cost?** Measured on more systems, in a machine-independent cost unit, counting a run as solved only if it reaches the right solution.
- Q5. What do the numerics say about its parameters?** The spectrum of the flat-start Jacobian, the feasible $(K, \Delta t_0)$ region, and tuning of the shift δ .

1.4 Contributions

- dynhomotopy, a Python package implementing NR, FDXB, a reconstruction of GSH-NR, the fixed-point homotopy with FE, RK2 and BE, and path schedules, all against a single $g(x)/J(x)$ interface.
- A complete reproduction of the paper (one script per table or figure, every cell printed as *ours* / *paper*) and an audit of eight inconsistencies in it.
- An algebraic explanation of the method: the two-point BE path is a shifted Newton step followed by a near-Newton step (Section 2.2).
- improvements, a package of eight switchable modifications evaluated in 17 configurations on 34 systems $\times$ 5 settings (2,890 runs), with LU counts as cost and a root check.
- The numerical studies of our proposal: power method and shifted inverse iteration for the Jacobian spectrum, $(K, \Delta t_0)$ feasibility maps, golden-section tuning of δ , and Gauss elimination and LU with partial pivoting written from scratch.

Code availability. All code, data download scripts, experiment drivers and the scripts that generate every figure and table of this report are in our public GitHub repository [21]: <https://github.com/JonayedMohiuddin/NumericalAnalysisProject>.

2 Methodology

2.1 The homotopy and its three integrators

With $\Phi(x, t) = -G_x(x, t)^{-1} G_t(x)$, the three schemes of the paper are

$$\text{FE: } x_{k+1} = x_k + \Delta t \Phi(x_k, t_k), \quad (6)$$

$$\text{RK2: } k_1 = \Phi(x_k, t_k), \quad k_2 = \Phi(x_k + \Delta t k_1, t_{k+1}), \\ x_{k+1} = x_k + \frac{\Delta t}{2} (k_1 + k_2), \quad (7)$$

$$\text{BE: } x_{k+1} = x_k + \Delta t \Phi(x_k, t_{k+1}). \quad (8)$$

The implicit equation of true backward Euler, $x_{k+1} = x_k + \Delta t \Phi(x_{k+1}, t_{k+1})$, is replaced by one fixed-point iteration started at x_k , as in the paper. What remains “implicit” about BE is only that it evaluates G_x at the *new* time t_{k+1} . Every evaluation of Φ costs one sparse LU factorization. Algorithm 1 gives the complete hybrid method.

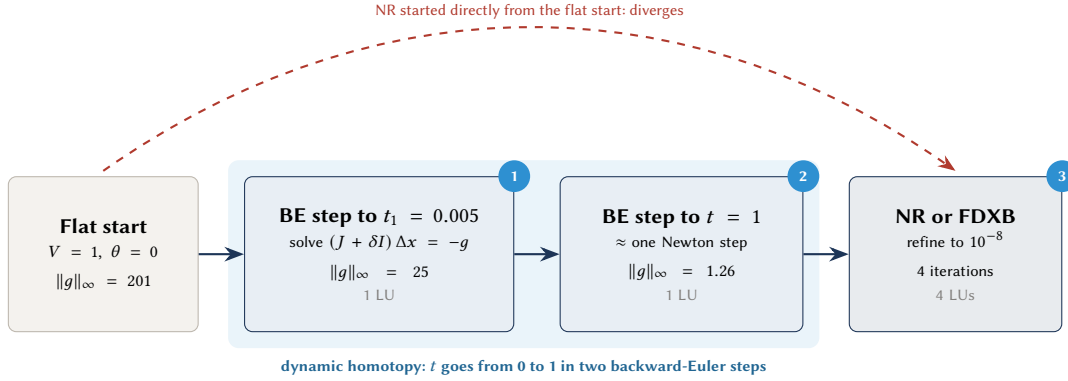


Figure 1: The base paper’s method at a glance, with the real mismatch norms of one solve of case109272. Stage 1 regularises the flat-start Jacobian by $\delta = K/\Delta t_0$; stage 2 is almost a plain Newton step; stage 3 is ordinary NR from a much better start (Section 2.2).

Algorithm 1 The base paper’s hybrid method

Input: (g, J) , flat start $x^{(0)}$, K , times $0 = t_0 < \dots < t_N = 1$, scheme $\in \{\text{FE, RK2, BE}\}$

- 1: $x \leftarrow x^{(0)}$
- 2: **for** $k = 0, \dots, N - 1$ **do**
- 3: $\Delta t \leftarrow t_{k+1} - t_k$; use BE if $k = 0$, else the chosen scheme
- 4: $x \leftarrow \text{STEP}(x, t_k, \Delta t)$ ▷ one LU per Φ
- 5: **if** a matrix is singular **or** $\|g(x)\|_\infty$ is not finite **then**
- 6: **return** failure
- 7: **end if**
- 8: **end for**
- 9: **return** NR(x) or FDXB(x)

2.2 What the first step really does

At $t_0 = 0$ the state is $x^{(0)}$, so $G_t = g(x^{(0)})$, and the schemes differ only in where they evaluate G_x .

Forward Euler uses $G_x(x^{(0)}, 0) = KI$, in which the network is invisible:

$$\Delta x_{\text{FE}} = -\frac{\Delta t_0}{K} g(x^{(0)}) = -50 g(x^{(0)}) \quad (K = 10^{-4}, \Delta t_0 = 0.005). \quad (9)$$

Every unknown is pushed by fifty times its own mismatch, and on the ill-conditioned systems the mismatch is in the hundreds.

Backward Euler uses $G_x(x^{(0)}, \Delta t_0) = \Delta t_0 J + (1 - \Delta t_0) KI$, so

$$\begin{aligned} \Delta x_{\text{BE}} &= -\Delta t_0 [\Delta t_0 J + (1 - \Delta t_0) KI]^{-1} g(x^{(0)}) \\ &= -[J + \delta' I]^{-1} g(x^{(0)}), \\ \delta' &= \frac{(1 - \Delta t_0) K}{\Delta t_0} \approx \frac{K}{\Delta t_0} = \delta = 0.02. \end{aligned} \quad (10)$$

This is a Newton step on g with the Jacobian shifted by δI , structurally a Levenberg–Marquardt step [14, 19]. The shift keeps the linear system solvable where the flat-start Jacobian is nearly singular, and shortens the step in the directions where J is weak, exactly where a plain Newton step overshoots.

The last step is almost Newton. The paper’s favourite path has only one more point, $t = 1$, where $G_x = J(x_1)$ and K disappears:

$$\begin{aligned} x(1) &= x_1 - (1 - \Delta t_0) J(x_1)^{-1} [g(x_1) - K(x_1 - x^{(0)})] \\ &\approx x_1 - J(x_1)^{-1} g(x_1). \end{aligned} \quad (11)$$

So the two-point BE “homotopy” is one shifted Newton step followed by an almost plain Newton step. This explains why its cost is essentially that of NR, why δ rather than K or Δt_0 alone is the parameter that matters, and why explicit schemes cannot compete with a large final step: at $t = 1$ they would use G_x evaluated at the previous, smaller t , still dominated by KI (Figure 2).

2.3 The refining solvers and the static baseline

NR stops when $\|g\|_\infty < 10^{-8}$ or after 10 iterations (MATPOWER’s defaults and the paper’s setting); a singular Jacobian or a non-finite mismatch is a failure. FDXB follows MATPOWER’s `fdpf`: B' ignores shunts, line charging, taps and resistance, B'' keeps the full network without phase shifters, and both are factorised once. It stops when both the P and the Q mismatch are below 10^{-8} , or after 100 P – θ updates. We count P updates plus Q updates, as the paper’s Table 7 does. *GSH-NR* [4] is described in the paper only in words, so ours is a reconstruction: fictitious shunts sized to make the flat start exact at $h = 0$ (only the active part at PV buses, none at the slack bus) and scaled by $(1 - h)$, with the resistance and reactance of every branch connected to the slack bus multiplied by $\delta + (1 - \delta)h$. h runs from Δh to 1 in steps of Δh , every point is a full NR solve (tolerance 10^{-8} , at most 10 iterations), and the reported count is the total number of NR iterations over all points. The pair $(\Delta h, \delta)$ is taken per system from the paper’s Table 7 (Table 2).

2.4 Modifications to the method

Every modification is a switch on top of the untouched reproduction. Building the reproduction first showed (Section 2.2) that the method is a shifted Newton step followed by a near-Newton step, so the Newton stages became the most promising place to intervene. Figure 3 shows where each switch acts.

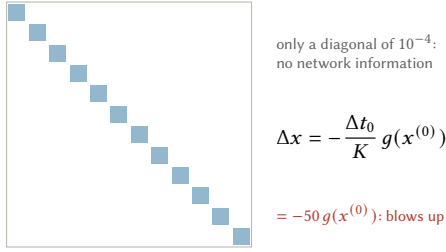
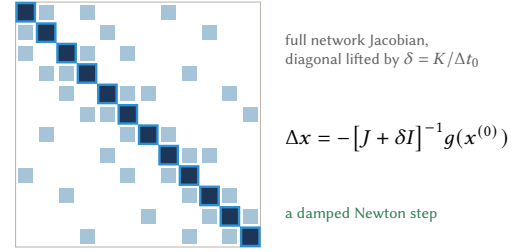
Forward Euler sees $G_x = KI$ Backward Euler sees $G_x \propto J + \delta I$ 

Figure 2: What each integrator “sees” on the first step. Forward Euler inverts KI , so its step ignores the network; backward Euler evaluates G_x one step later, where it is proportional to the real Jacobian plus a small diagonal shift.

M1: optimal multiplier in NR. Iwamoto and Tamura [10] scale each Newton step Δx by μ . With $a = g(x)$ and $c = g(x + \Delta x)$, the mismatch along the step is modelled as $g(x + \mu\Delta x) \approx (1 - \mu)a + \mu^2 c$, exact when g is quadratic (rectangular coordinates). Minimising $\|(1 - \mu)a + \mu^2 c\|_2^2$ gives the cubic

$$2(c^\top c)\mu^3 - 3(a^\top c)\mu^2 + (a^\top a + 2a^\top c)\mu - a^\top a = 0, \quad (12)$$

of which we take the real root in $(0, 2]$ with the smallest model mismatch. It costs no extra LU, and near the solution $\mu \rightarrow 1$, so quadratic convergence is kept.

M2: Newton corrector after every step. As in predictor–corrector continuation [1], after any step to t_{k+1} we take one Newton step on $G(x, t_{k+1}) = 0$: $x_{k+1} \leftarrow x_{k+1} - G_x(x_{k+1}, t_{k+1})^{-1} G(x_{k+1}, t_{k+1})$, at one extra LU per path point. The corrector is applied after every step, the first BE step included. At $t = 1$ it is exactly one NR step on g .

M3: Richardson error control of the path steps. After the BE step to $t_1 = \Delta t_0$, each attempt from t with step Δt computes one full step x_{full} and two half steps x_{half} (step doubling [6]). For a rule of order p ,

$$\varepsilon = \frac{\|x_{\text{half}} - x_{\text{full}}\|_\infty}{2^p - 1}, \quad x_{\text{new}} = x_{\text{half}} + \frac{x_{\text{half}} - x_{\text{full}}}{2^p - 1}, \quad (13)$$

$$\Delta t \leftarrow \Delta t \cdot \min\left(4, \max(0.1, 0.9(\text{tol}/\varepsilon)^{1/(p+1)})\right). \quad (14)$$

The step is accepted if $\varepsilon \leq \text{tol} = 0.5$ (radians and per unit), and the extrapolated x_{new} is kept; if $g(x_{\text{new}})$ is not finite, the two-half-step result is kept instead. A step that breaks down (singular matrix or non-finite mismatch) is retried at a quarter of its size, and a path that needs more than 150 LUs is stopped and counted as a failure (Algorithm 2). Every attempt costs three steps. The step rule can be BE ($p = 1$), FE ($p = 1$), RK2 ($p = 2$) or RK4 ($p = 4$); the first step to t_1 is always BE. The large tolerance is deliberate: the path only has to end inside NR’s region of convergence.

M4: adaptive time points by mismatch. A cheaper rule keeps the first step to t_1 , then tries to jump straight to $t = 1$. A step is kept if it reduced $\|g\|_\infty$ and otherwise halved (at most six times); after an accepted step the next one is doubled. Rejected trials still count as LUs.

M5–M8: four further ideas. M5, BE-chord: three fixed-point iterations $\hat{x} \leftarrow x_k - \Delta t G_x(x_k, t_{k+1})^{-1} G_t(\hat{x})$, started at $\hat{x} = x_k$ and reusing one LU. The paper writes BE as the fixed-point iteration of

Algorithm 2 Richardson-controlled path

```

1:  $x \leftarrow \text{BE}(x^{(0)}, 0, \Delta t_0)$ ;  $t \leftarrow \Delta t_0$ ;  $\Delta t \leftarrow 1 - \Delta t_0$ 
2: while  $t < 1$  and at most 150 LUs used do
3:    $\Delta t \leftarrow \min(\Delta t, 1 - t)$ ;  $x_f \leftarrow \text{STEP}(x, t, \Delta t)$ 
4:    $x_h \leftarrow \text{STEP}(\text{STEP}(x, t, \frac{\Delta t}{2}), t + \frac{\Delta t}{2}, \frac{\Delta t}{2})$ 
5:   if a step broke down then  $\Delta t \leftarrow \Delta t/4$ ; continue
6:   end if
7:    $\varepsilon \leftarrow \|x_h - x_f\|_\infty / (2^p - 1)$ 
8:   if  $\varepsilon \leq \text{tol}$  then  $x \leftarrow x_h + (x_h - x_f)/(2^p - 1)$ ;  $t \leftarrow t + \Delta t$ 
9:   end if
10:   $\Delta t \leftarrow \Delta t \cdot \min(4, \max(0.1, 0.9(\text{tol}/\varepsilon)^{1/(p+1)}))$ 
11: end while

```

its eq. (19) and uses one iteration; M5 tests whether more iterations help. It replaces BE on every step, the first included. **M6, RK4** (classical fourth order, four LUs per step) with fixed steps after a BE first step (the paper’s future work). **M7, scaled homotopy:** $KD(x - x^{(0)})$ with $D = |\text{diag } J(x^{(0)})|$ normalised to mean one (zero diagonal entries replaced by the smallest nonzero one), in the spirit of [3]. **M8, Newton homotopy:** $G(x, t) = g(x) - (1 - t)g(x^{(0)})$ [28], which has $G_x = J(x)$ and no KI term. M7 and M8 are followed with BE.

2.5 Numerical studies from our proposal

Spectrum of the flat-start Jacobian. The first BE step solves with $J + \delta I$, whose eigenvalues are those of J moved by δ . The largest eigenvalue modulus comes from the power method [5]; because J is non-symmetric and its dominant eigenvalues may form a complex or $\pm\lambda$ pair, we iterate with J applied twice and take $\sqrt{\|J^2 x\|/\|x\|}$. The smallest modulus comes from shifted inverse iteration: the power method on $(J - \sigma I)^{-1}$ using one sparse LU of $J - \sigma I$, with $\sigma = 0$ for J and $\sigma = -\delta$ for $J + \delta I$. ARPACK [13] (via SciPy) is an independent check.

Feasibility maps. The paper’s method (BE on $\{0, \Delta t_0, 1\}$, then NR) is run on an 8×11 grid, $\Delta t_0 \in [0.001, 0.2]$ and $K \in [10^{-6}, 10^{-1}]$, recording whether each run reaches the reference solution, another root, or fails.

Golden-section tuning of δ . The objective is the total LU count (a failure costs 50) plus a tie-break $0.1 \log_{10} \|g(x(1))\|_\infty$. Golden-section search [11] runs over $\log_{10} \delta \in [-4, 1]$ at $\Delta t_0 = 0.005$, then over $\log_{10} \Delta t_0 \in [-3, -0.5]$, each to a bracket of 0.05 decades (21

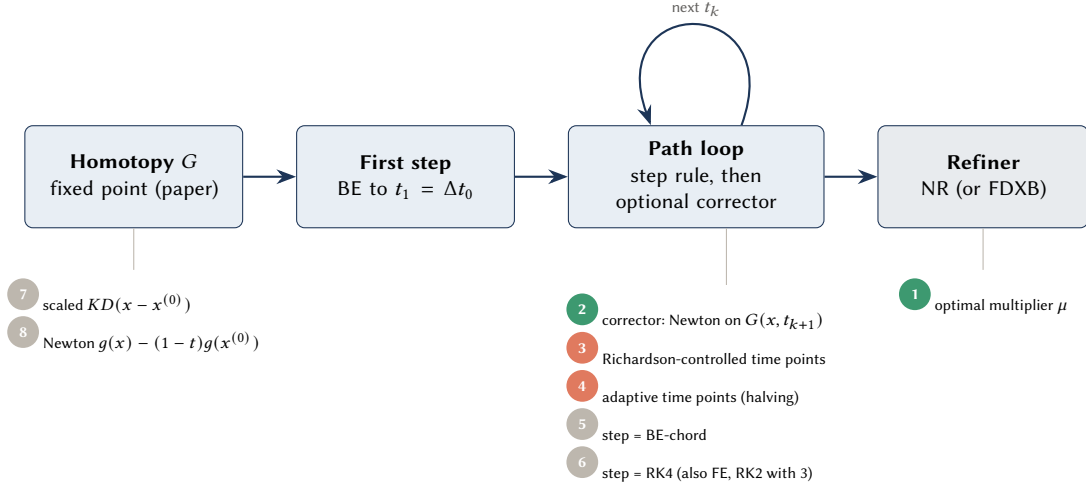


Figure 3: Where each of the eight modifications acts. Green: the two that help on their own and together; orange: the two path-control rules, which help on some systems at a price; grey: the four that did not work (Section 4.3).

solves per system). Since the objective is piecewise constant and not unimodal, the best point seen is returned.

Linear solvers from scratch. Gauss elimination with partial pivoting and back substitution, and Doolittle LU with partial pivoting ($PA = LU$, L and U in one array, permutation as an index vector) with forward and back substitution. Both are dense and $\mathcal{O}(n^3)$; the paper’s BE path, NR and FDXB are repeated with every linear system solved this way.

3 Implementation and Experiments

3.1 Software architecture

Everything in `dynhomotopy` is written against one interface: a problem supplies $g(x)$ and $J(x)$, nothing else, so the paper’s 2×2 tutorial problem and the 109,272-bus grid go through the same solver and integrator code (Figure 4). The power flow *model*—reading a MATPOWER case, renumbering buses, building Y_{bus} , S_{bus} , B' , B'' and the derivatives $\partial S / \partial V$ for the Jacobian—uses PYPOWER [18], a Python port of MATPOWER [29], which contains only formulas, no solving. Every numerical method (NR, FDXB, GSH-NR, the homotopy, the integrators, the LU counter and everything in improvements) is our own code, built on NumPy [8] and SciPy [27]. Sparse linear systems are solved with SuperLU [15] through SciPy, and every factorization in the program goes through one counter, which is how cost is measured. Figures are drawn with Matplotlib [9]. The modifications live in a second package that imports the first and never edits it; a test asserts that with every switch off the modified solver returns the paper’s method bit for bit. Listing 1 shows the core of the method.

3.2 Datasets and benchmarks

All systems are MATPOWER cases from two public Zenodo records by Tostado-Véliz, Kamel and Jurado, the same files the paper uses: record 3514739 holds the *ill-conditioned* systems [24], built by gluing copies of European PEGASE [2] and Polish networks together,

Listing 1: The core of the method (`homotopy/fpv.py`, `homotopy/integrators.py`), lightly trimmed.

```
class FixedPointHomotopy:
    # G = t g(x) + (1-t) K (x - x0)
    def G(self, x, t):
        return t*self.problem.g(x) + (1-t)*self.K*(x - self.x0)
    def Gx(self, x, t):
        return (t*self.problem.jacobian(x)
                + (1-t)*self.K*self.eye).tocsc()
    def Gt(self, x):
        return self.problem.g(x) - self.K*(x - self.x0)
    def dxdt(self, x, t):
        # one sparse LU per call, counted
        return -solve(self.Gx(x, t), self.Gt(x), self.counter)

    def forward_euler(h, x, t, dt):
        return x + dt*h.dxdt(x, t)
    def backward_euler(h, x, t, dt):
        return x + dt*h.dxdt(x, t+dt)
    def runge_kutta2(h, x, t, dt):
        k1 = h.dxdt(x, t); k2 = h.dxdt(x + dt*k1, t + dt)
        return x + dt/2*(k1 + k2)
```

and record 3491654 the *loading-limit* systems [25], in which every load of a standard case was scaled up until NR from a flat start was about to stop converging. The code downloads each file on first use. The paper uses nine of these systems; for the modification study we added every other system in the two records that loads, giving 34 (Table 1 and Figure 5). Twenty-five of them were never seen while the paper’s parameters were chosen, so they test whether those parameters generalise. The replicated construction is visible in the Jacobian (Figure 19, Section C): case54636 and case109272, four and eight copies of the same network, behave *identically* in every experiment, which is a useful check on the paper (Section 4.2). Throughout the report case500limit and case2000limit are short for the files case_ACTIVSg500limit and case_ACTIVSg2000limit; the paper uses both forms.

Four details that change numbers. (i) case69limit and case141limit are stored with baseMVA = 10, but the paper’s Tables 2–5 come out only on 100 MVA, so we use 100 (Section 4.2). (ii) The flat start includes the slack angle; for NR-MAT (NR from the stored operating point) we shift the stored angles so the slack is

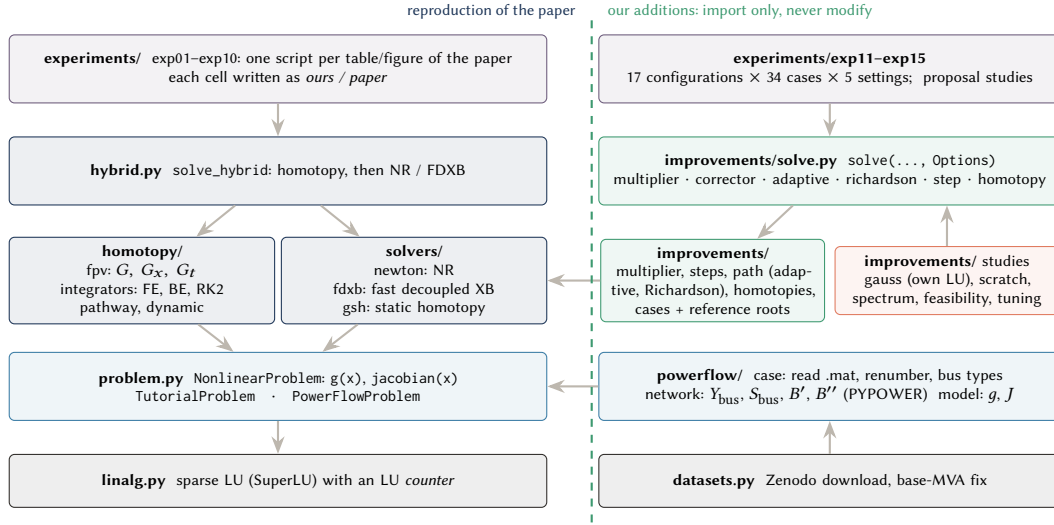


Figure 4: Architecture of the implementation. Every solver sees only $g(x)$ and $J(x)$, and every LU factorization in the program goes through one counter.

at zero. (iii) Six of the 22 limit cases converge from the flat start in *exactly* ten NR iterations and eleven more fail at ten, so any change worth one NR iteration can flip a limit case. (iv) The power flow equations have several roots, and a mismatch below 10^{-8} does not say which one was found. For every system the *reference* solution is the one NR reaches from the initial guess stored in the case file (up to 50 iterations, with the optimal multiplier as a fallback); a run is *solved* only if every bus voltage is within 10^{-4} pu of it.

3.3 Experimental setup and parameter choices

Reproduction. Each result of the paper has its own script (exp01–exp10, Table 10). Unless stated otherwise $K = 10^{-4}$ and $\Delta t_0 = 0.005$ ($\delta = 0.02$), NR tolerance 10^{-8} with 10 iterations, and the paths are those of the paper: $\{0, 0.005, 0.01, 0.51, 1\}$ for Tables 2–4, $\{0, 0.005, 0.01, 0.02, 0.12, 0.62, 1\}$ for Table 5, $\{0, 0.05, 0.1, t_3, 1\}$ with $K = 10^{-3}$ for Figure 3 and Table 6, and $\{0, 0.005, 1\}$ for BE in Table 7. Timings are medians of seven repetitions.

Modification study. Every configuration runs on all 34 systems under five settings of $(\Delta t_0, K)$, all on the path $\{0, \Delta t_0, 1\}$: S1 ($0.005, 10^{-4}$) the paper’s setting, S2 ($0.05, 10^{-3}$) and S3 ($0.1, 2 \times 10^{-3}$) from the paper’s Section 4.4, S4 ($0.005, 10^{-5}$) a weak K , and S5 ($0.005, 10^{-3}$) a strong K . M1, M2 and M4 run in all eight on/off combinations; M3 with each of BE, FE, RK2 and RK4 and together with M1; and M5–M8 alone: 17 configurations, 2,890 runs. Cost is the number of LU factorizations, which does not depend on the machine. If a configuration solves a run the paper’s method does not, the paper’s method is re-run with 30 NR iterations to classify the gain as a *speed-up* (it then reaches the reference), a *root fix* (it reaches another root) or a *rescue* (it still diverges).

Parameter summary. Table 2 lists every parameter of the methods and studies, with the value used and where it comes from. All values are read from the code.

Verification and software. Before any comparison, the building blocks are checked by 24 automated tests (pytest [12]), all of which pass on the current code: the analytic Jacobian against finite differences on case691limit and case500limit; the path generator against the paper’s time grids; the BE first step against the paper’s linearised static homotopy (eqs. 25 and 27, to 10^{-10}); NR on the tutorial in exactly the paper’s 11 iterations; the NR-MAT and NR-flat counts, a full row of Table 4 and the FDXB count of Table 7 for case2000limit; the switch-off identity; the optimal multiplier, the corrector, the adaptive rule, the explicit rules’ BE first step and both alternative homotopies; our Gauss elimination and LU against NumPy, with pivoting past a zero diagonal, and with the same Newton iterates as SuperLU on case500limit; the eigenvalue routines on a diagonal matrix; golden-section search on a parabola; Richardson control reaching $t = 1$; and both command-line tools. The software dependencies are those of the repository’s requirements.txt: NumPy ≥ 1.24 , SciPy ≥ 1.10 , Matplotlib ≥ 3.7 , PYPOWER ≥ 5.1 and pytest ≥ 7 .

4 Results and Discussion

4.1 Reproducing the base paper

The two-variable tutorial. The paper first tests the idea on $g_1 = x_1^2 + x_2^2 - 2x_1x_2 - 1$, $g_2 = x_1 + x_2 - 2$ from $x^{(0)} = (1, 1)$, where the Jacobian is singular. The system has two roots, $A = (1.5, 0.5)$ and $B = (0.5, 1.5)$, and the singular set is the line $x_1 = x_2$ through the start. Figure 6 draws what the integrators do in this plane, which the paper shows only as mismatch curves: with $K = 0.05$ all three stay near the exact path; with $K = 0.005$ the explicit ones overshoot across the singular line and NR then converges to root B , while BE reaches A . Table 3 and Figure 20 (Section C) compare iteration counts: NR alone matches the paper for $\epsilon = 0.005$ and 0.01 ; BE is the best start for NR in every hybrid panel, as in the paper, though some counts differ by one or two.

Table 1: The 34 test systems. n is the number of unknowns, nnz the nonzeros of the Jacobian at the flat start, and the last column the NR iterations needed from the flat start (limit 10).

Case	Built from	Buses	PV	n	nnz(J)	$\ g(x^{(0)})\ _\infty$	NR flat
<i>Paper, ill-conditioned (record 3514739)</i>							
case18482	9241pegase x2	18,482	2,889	34,073	258,947	533	fail
case27318	13659pegase x2	27,318	8,183	46,451	349,411	201	fail
case36964	9241pegase x4	36,964	5,779	68,147	517,923	533	fail
case54636	13659pegase x4	54,636	16,367	92,903	698,829	201	fail
case109272	13659pegase x8	109,272	32,735	185,807	1,397,667	201	fail
<i>Paper, loading limit (record 3491654)</i>							
case691limit		69	0	136	808	0.0124	2
case141limit		141	0	280	1,672	6.4e-03	2
case500limit		500	55	943	6,275	13.3	4
case2000limit		2,000	391	3,607	26,345	55	5
<i>Added by us, ill-conditioned (record 3514739)</i>							
case6024	3012wp x2	6,024	595	11,451	72,553	818	fail
case6243	3375wp + 2869pegase	6,243	901	11,583	77,335	1.45e+03	fail
case6748	3375wp x2	6,748	783	12,711	81,461	1.45e+03	fail
case7092	2869pegase x2 + 1354pegase	7,092	1,279	12,903	89,043	562	fail
case9961	2869pegase x3 + 1354pegase	9,961	1,789	18,131	125,663	562	fail
case10595	9241pegase + 1354pegase	10,595	1,704	19,484	145,244	562	fail
case12110	9241pegase + 2869pegase	12,110	1,954	22,264	166,032	559	fail
<i>Added by us, loading limit (record 3491654)</i>							
case141limit		14	4	22	146	3.75	fail
case_ieee30limit		30	5	53	379	2.77	10
case571limit		57	6	106	718	3.81	10
case89pegaselimit		89	11	165	1,811	324	fail
case118limit		118	53	181	1,051	5.89	10
case300limit		300	68	530	3,736	25.8	10
case1354pegaselimit		1,354	259	2,447	15,803	562	fail
case2383wplimit		2,383	326	4,438	27,874	9.79	10
case2736splimit		2,736	233	5,237	33,715	438	fail
case2737soplmit		2,737	192	5,280	34,242	761	fail
case2746woplmit		2,746	349	5,141	32,445	512	fail
case2746wplimit		2,746	363	5,127	32,129	513	fail
case2869pegaselimit		2,869	509	5,227	36,591	559	10
case3012wplimit		3,012	297	5,725	36,263	818	fail
case3120splimit		3,120	247	5,991	38,329	611	fail
case3375wplimit		3,374	391	6,355	40,717	1.45e+03	fail
case9241pegaselimit		9,241	1,444	17,036	129,412	532	fail
case13659pegaselimit		13,659	4,091	23,225	174,703	201	8

The first time step. Figure 7 measures the paper’s Section 4.2.1 on all nine systems: with $K = 10^{-4}$, FE multiplies the mismatch by factors between 7×10^3 (case691limit) and 1.8×10^{10} (case18482), and RK2, which evaluates its second slope at the FE point, by factors between 1.9×10^3 and 1.4×10^{10} . BE reduces the mismatch by about a factor of eight on every ill-conditioned system (533 to 65 or 66, 201 to 25), the values the paper prints in the $t = 0.005$ column of its Table 4, and it also reduces it on all four limit cases. Taking the FE step with $K = 1$, the value earlier work used, makes the blow-up much smaller but does not remove it: the mismatch still grows on all nine systems, for example from 533 to 1.3×10^5 on case18482 and from 55 to 631 on case2000limit.

Tables 2–5: mismatch along the path. On the paper’s path $\{0, 0.005, 0.01, 0.51, 1\}$ our BE results (Table 4) match the paper almost digit for digit, including the three NR iterations that follow; the FE and RK2 tables (Tables 11 and 12, Section B) match wherever the schemes stay finite and blow up at the same point where they diverge. With the growing steps of Table 5 (Table 14, Section C) all three schemes give NR a usable start. Figure 8 puts all 255 numeric cells of Tables 2–5 on one plot: 246 (96%) agree within a factor of two and 200 (78%) within 10%, the rounding of the paper’s two

printed digits. Of the nine outliers, seven are divergent FE/RK2 values above 10^3 , one is a Table 2 cell discussed in Section 4.2, and one is a final NR mismatch near 10^{-11} .

Section 4.4 and Figure 3: larger first steps. The paper raises Δt_0 to 0.05 and K to 0.001 and reports that with few points only BE works. In our full grid (Figure 9) BE gives NR a usable start in 75 of 81 (path, system) cells, FE in 64 and RK2 in 59; on three-point paths BE works on 8 of 9 systems and FE and RK2 on only 3, confirming the main claim. With five-point paths (Figure 21, Section C) we reproduce the paper’s Figure 3 including its failure (FE, $t_3 = 0.25$, case36964), and find one more: RK2 with $t_3 = 0.20$ on the same system. The Table 6 timings on case109272 are within 13 percentage points of the paper in every row (Table 13, Section B). On case36964 FE is within 22 points and RK2 and BE are 22–32 points above the paper. On case2000limit we measure 237–342% against the paper’s 61–94%; we cannot explain a hybrid run being faster than NR-MAT there, since it does strictly more work (four homotopy LUs plus three NR iterations against NR-MAT’s three iterations).

Figures 4 and 5: the states themselves. Following five bus voltages of case109272 through a two-point BE path and NR (Figure 10),

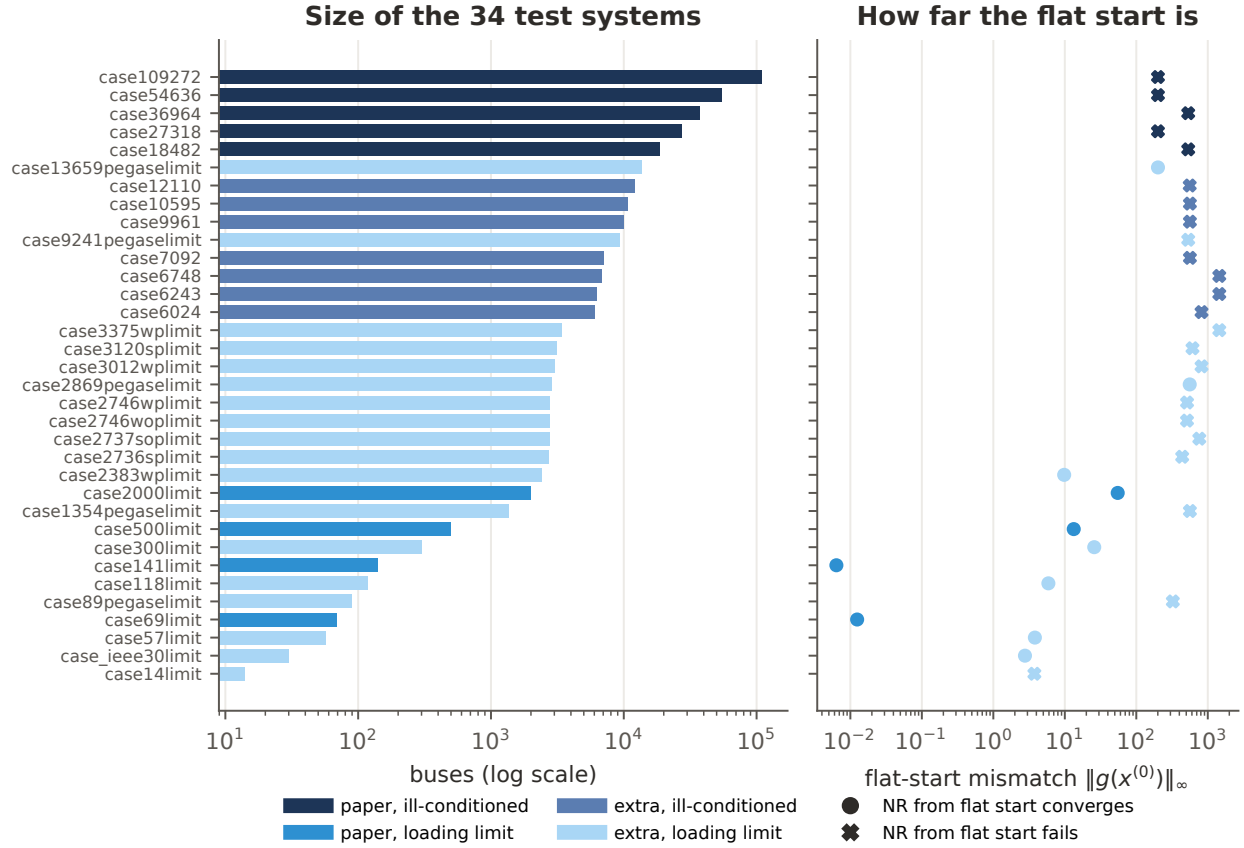


Figure 5: Size and flat-start mismatch of all 34 systems. Size alone does not decide difficulty: NR from the flat start fails on the 14-bus limit case and succeeds on the 13,659-bus one. The failing systems share a large initial mismatch.

angles overshoot to about 60 degrees at $t = 1$ and NR pulls them back within one iteration; bus 6 ends at 1.0045 pu and 21.10 degrees. The mismatch goes $201 \rightarrow 25 \rightarrow 1.26$ along the path and $1.70 \rightarrow 1.7 \times 10^{-2} \rightarrow 7.3 \times 10^{-7} \rightarrow 8.5 \times 10^{-12}$ in NR: four iterations, as the paper reports. Varying the constant step after t_1 (Figure 22, Section C), smaller steps give NR a closer start and save one NR iteration, but each extra path point costs an LU, so total cost rises from 6 to 14 factorizations (Figure 11), which quantifies the paper’s own conclusion.

Table 7: iterations. Table 5 is the paper’s headline comparison. NR from the flat start fails on all five ill-conditioned grids; BE(NR) needs 4 iterations and RK2(NR) 3 on all of them, and FDXB counts match to the iteration. 54 of the 63 cells agree exactly. Six of the nine differences are the two small limit cases, whose counts match the paper only on the stored 10 MVA base; the others are our GSH-NR reconstruction on two systems and one FDXB count for case109272 (Section 4.2). Every converged run in the table reaches the reference operating point.

Table 8: run time. The timing results of Tables 6 and 8 are the medians of seven runs saved in the repository; rerunning both experiments with the current code gives identical iteration counts and failures. Relative to NR-MAT (Figure 12), BE(FDXB) is faster

than NR-MAT on all five ill-conditioned systems, at 66–93% of its time against the paper’s 57–85%, and BE(NR) costs slightly more (102–122% against 92–128%). Both conclusions of the paper hold. RK2 is much slower for us (252–331%) than for the paper (84–218%): its path has seven steps after $t = 0$ and every step needs two LUs, one per slope; we take that, rather than a property of Python, as the likely reason, but did not profile it. Because Python and MATLAB timings are not comparable, our own cost comparisons use LU counts.

Reproduction (Q1, Q2). *The method works as the paper says. On all five ill-conditioned grids, a two-point BE path turns a divergent flat start into a four-iteration NR solve, and our numbers track the paper’s across 28 orders of magnitude. Where explicit schemes fail, they fail where the paper says, for the reason Section 2.2 gives.*

4.2 Auditing the base paper

For every disagreement we first checked our own code; the eight in Table 6 survived that check. Two further conventions had to be discovered before anything matched: FDXB iterations are counted as P plus Q updates, and the flat start includes the slack angle. With those fixed, the base-MVA issue is the only one that changes a conclusion, and only for the two smallest systems.

Table 2: Parameters of the methods, modifications and studies, as set in the code.

Parameter	Value	Source
Homotopy factor K , first step Δt_0	10^{-4} , 0.005 ($\delta = K/\Delta t_0 = 0.02$); 10^{-3} with $\Delta t_0 = 0.05$ and 2×10^{-3} with $\Delta t_0 = 0.1$ in Section 4.4, Figure 3 and Table 6	base paper
NR	tolerance $\ g\ _\infty < 10^{-8}$, at most 10 iterations (30 to classify gains, 50 for the reference solution)	paper / MATPOWER defaults
FDXB	tolerance 10^{-8} on P and Q mismatch, at most 100 P - θ updates	MATPOWER fdcpf
GSH-NR ($\Delta h, \delta$)	case18482 (0.5, 1); case27318 (0.25, 0.125); case36964 (0.25, 1); case54636, case109272 (0.01, 0.01); the four limit cases (1, 1)	paper, Table 7
Tutorial	$x^{(0)} = (1, 1)$, $\Delta t_0 = 0.01$, then constant $\Delta t \in \{0.5, 0.125\}$, $K \in \{0.05, 0.01, 0.005\}$, NR tolerance 10^{-5}	paper, Section 3.3
Paths for Tables 7 and 8	BE: $\{0, 0.005, 1\}$; RK2: $\{0, 0.005, 0.01, 0.02, 0.12, 0.42, 0.72, 1\}$	paper
Reference solution	NR from the stored initial guess, up to 50 iterations, optimal multiplier as fallback; a run is on the reference if $\max_k V_k - V_k^{\text{ref}} < 10^{-4}$ pu	ours
Richardson control (M3)	tol = 0.5, step factor $\min(4, \max(0.1, 0.9(\text{tol}/\varepsilon)^{1/(p+1)}))$, retry at $\Delta t/4$ after a breakdown, at most 150 LUs per path	ours
Adaptive steps (M4)	at most 6 halvings per step, next step doubled after an accepted one	ours
BE-chord (M5)	3 fixed-point iterations per step with one LU	ours
Optimal multiplier (M1)	real root of the cubic in $(0, 2]$ with the smallest model mismatch, else $\mu = 1$	Iwamoto and Tamura
Power method / inverse iteration	random start (seed 0), relative tolerance 10^{-6} , at most 500 / 200 iterations; shift $\sigma = -0.02$ for $J + \delta I$	ours
Feasibility grid	$\Delta t_0 \in \{0.001, 0.0025, 0.005, 0.01, 0.025, 0.05, 0.1, 0.2\}$, 11 values of K log-spaced in $[10^{-6}, 10^{-1}]$, path $\{0, \Delta t_0, 1\}$; systems case18482, case36964, case6024, case2000limit	ours
Golden-section tuning	$\log_{10} \delta \in [-4, 1]$ at $\Delta t_0 = 0.005$, then $\log_{10} \Delta t_0 \in [-3, -0.5]$; bracket 0.05, at most 25 evaluations per search; failure costs 50 LUs; tie-break $0.1 \log_{10} \ g(x(1))\ _\infty$	ours

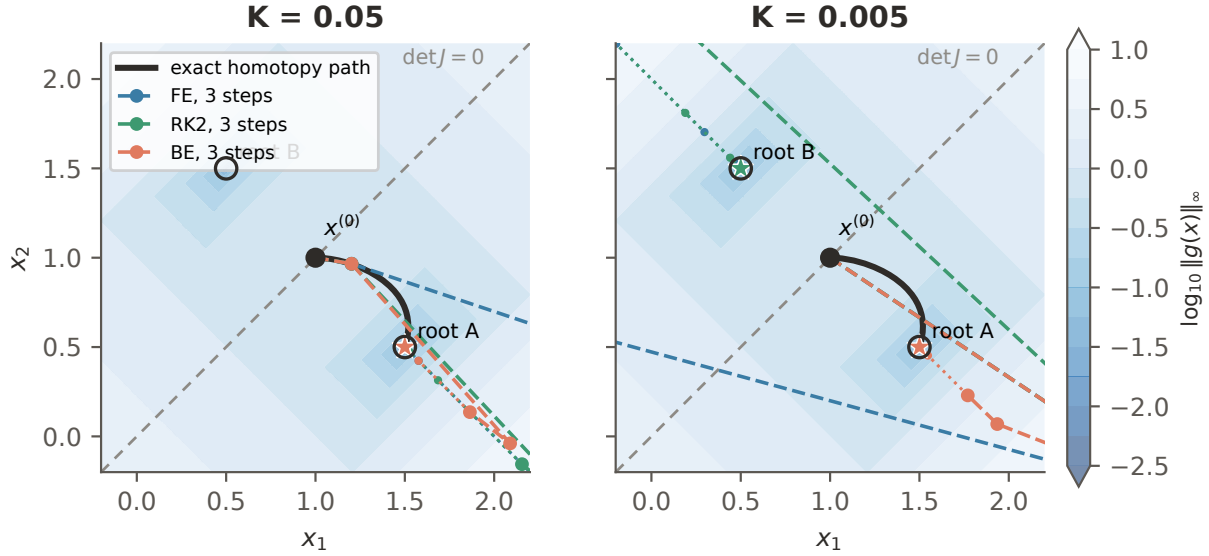


Figure 6: The tutorial in the (x_1, x_2) plane, $\Delta t_0 = 0.01$ then $\Delta t = 0.5$. Background: $\log_{10} \|g\|_\infty$. Black: the exact homotopy path; dashed: the three coarse integrations; dotted: the NR iterations; stars: where NR ends. At the smaller K the explicit schemes cross $\det J = 0$ and NR finds the other root.

4.3 Results of the modifications

First, which root? On the 170 runs of the test bed, the paper’s method gives NR a start from which it converges 141 times. But 14 of those runs converge to a *different* solution than the reference: case54636 and case109272 in S3, S4 and S5, case27318 in S4 and S5, case3012wplimit in S1 and S2, case3375wplimit in S1, S2 and S3, and case13659pegaselimit in S4. The mismatch is below 10^{-8} in all of them; only comparing the voltages tells them apart. In the paper’s own setting S1 all nine of the paper’s systems reach the

reference, and so does every converged run of our Table 7 re-run, which checks it; but on the wider test bed the method truly solves 127 of 170 runs, not 141.

Headline numbers. Table 7 and Figure 13 give all 2,890 runs. Every configuration that contains the corrector solves 152 and loses nothing; the multiplier alone solves 143; Richardson control solves 145–158 depending on the step rule. The four other ideas lose runs, fixed-step RK4 catastrophically. Figure 14 plots robustness against cost: multiplier plus corrector is both more robust *and* cheaper than

Table 3: Tutorial: NR iterations to $\|g\|_\infty < 10^{-5}$, ours and read off the paper’s plots, and the root reached.

Run	Δt	K	Ours	Paper	Root
NR, $\epsilon = 0.005$	–	–	11	11	A
NR, $\epsilon = 0.01$	–	–	10	10	A
NR, $\epsilon = 0.05$	–	–	8	9	A
FE/BE/RK2	0.5	0.05	6/4/4	6/3/6	A/A/A
FE/BE/RK2	0.5	0.01	6/2/4	4/2/4	B/A/B
FE/BE/RK2	0.5	0.005	8/4/7	6/3/7	B/A/B
FE/BE/RK2	0.125	0.05	3/3/2	3/2/3	A/A/A

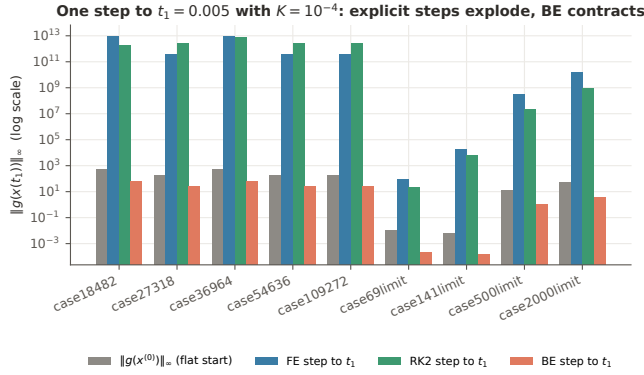


Figure 7: Mismatch after the single first step to $t_1 = 0.005$, $K = 10^{-4}$ (log scale). Grey is the flat start. FE and RK2 make every system worse by orders of magnitude; BE improves every one.

the paper’s method; the corrector alone is as robust at about 5% more LUs; Richardson control costs 18–30% more with BE and 4–8 times more with FE, RK2 and RK4.

The corrector is the robust improvement, the multiplier the cheap one (Q4). The Newton corrector alone solves 25 more runs than the paper’s method (2 rescues, 13 wrong roots corrected, 10 speed-ups), never loses one, and cuts the other-root runs from 14 to 2, for 0.4 more LUs per system in S1 (8.43 against 8.04). The optimal multiplier alone is 11% cheaper (7.14 LUs) but still lands on another root 12 times. Together, multiplier plus corrector keeps all of the corrector’s robustness (152 solved, no losses) and most of the multiplier’s saving (7.57 LUs in S1, 7% fewer than the paper’s method over all settings). It is the configuration we recommend.

Richardson step control. Error-controlled steps are the only change that solves some runs nothing else solves: case6024 in S1 (73 LUs with BE), and case7092, case10595 and case12110 with the weak K of S4. Multiplier plus Richardson rescues 10 of the 11 divergent runs of the test bed, more than any other configuration, and solves 156 runs without losing one. On case6024 (Figure 15) the paper’s jump from t_1 to $t = 1$ lands where NR cannot recover; the controller forces small steps early, where the state changes fastest, and lets them grow later. The study also tests the paper’s preference for BE under fair, error-controlled conditions, and it holds: in S1 Richardson control needs 10.6 LUs with BE, 37 with FE, 48 with RK2 and 87 with RK4.

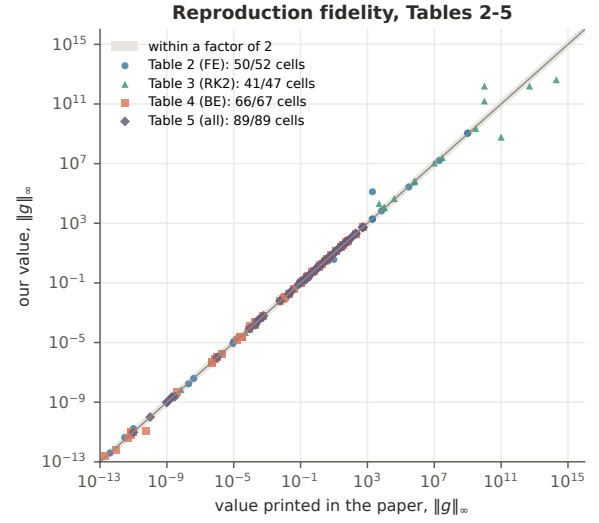


Figure 8: Every numeric cell of the paper’s Tables 2–5 against our value, on log axes spanning 28 orders of magnitude. The grey band is agreement within a factor of two.

System by system, and the nature of the gains. On the paper’s nine systems little changes in S1 (Figure 23, Section C): the method already works with 3–6 LUs. The differences are on the added systems: the multiplier saves one to three LUs on 13 of the 14 extra limit cases that both it and the paper’s method solve, and on case3012wplimit and case3375wplimit every configuration with the corrector reaches the reference (9–11 LUs) where the paper’s method finds another root. Of the 36 runs the paper’s method does not solve but some configuration does, 10 are *speed-ups* (case141limit and case2736splimit in all five settings: the paper’s method needs 11 NR iterations, one over budget), 15 are *root fixes* (13 of them by the corrector; the other two, case109272 and case13659pegaselimit in S4, by Richardson control with an explicit step rule, and the latter also by BE-chord) and 11 are *true rescues* (the paper’s method diverges even with 30 NR iterations), of which multiplier plus Richardson rescues 10, Richardson with BE 9, multiplier plus adaptive 4, the corrector 2 and the multiplier 1 (Figure 24). The adaptive rule solves case6748, which the paper’s fixed path cannot, by halving the jump six times and then doubling its way to $t = 1$ (Figure 25); both runs are drawn in Section C.

4.4 Results of the numerical studies

Spectrum. On the twelve ill-conditioned systems $|\lambda|_{\min}(J(x^{(0)}))$ is 4×10^{-4} to 6×10^{-3} while $|\lambda|_{\max}$ is $2.7\text{--}5.2 \times 10^4$, so the eigenvalue ratio reaches 1.2×10^8 (case109272). The paper’s $\delta = 0.02$ lifts the smallest eigenvalue to 0.020–0.026 and cuts the ratio by a factor of 4 to 52, to $1.1\text{--}2.3 \times 10^6$ on all twelve (Figure 16). On 20 of the 22 limit cases the smallest eigenvalue is already 0.012 to 1.9, comparable to the shift or larger, so the shift changes their spectrum little. The two exceptions are the limit cases built from the two largest PEGASE grids, case9241pegaselimit (4.9×10^{-3}) and case13659pegaselimit (4.8×10^{-4}), whose spectrum looks like that of the ill-conditioned systems. Inverse iteration agrees with

Table 4: Table 4 re-run: $\|g\|_\infty$ along the BE path $\{0, 0.005, 0.01, 0.51, 1\}$ and after each of the first three NR iterations, $K = 10^{-4}$. Each case shows our value above the paper's.

Case		$t=0$	0.005	0.01	0.51	1	NR 1	NR 2	NR 3	Result
case18482	ours	533	65	33	7.2	3.7	0.042	1.5×10^{-5}	9.3×10^{-12}	conv. (3)
	paper	532	65	33	7.2	3.7	0.04	1.5×10^{-5}	10^{-11}	
case27318	ours	201	25	13	0.5	0.25	0.096	2.5×10^{-5}	6.6×10^{-12}	conv. (3)
	paper	200	25	13	0.5	0.25	0.1	2.5×10^{-5}	7×10^{-12}	
case36964	ours	533	66	33	6.9	3.6	0.27	6.5×10^{-4}	4.5×10^{-9}	conv. (3)
	paper	532	65	33	6.9	3.6	0.27	6×10^{-4}	4×10^{-9}	
case54636	ours	201	25	13	0.5	0.25	0.096	2.5×10^{-5}	9.4×10^{-12}	conv. (3)
	paper	200	25	13	0.5	0.25	0.1	3×10^{-5}	7×10^{-12}	
case109272	ours	201	25	13	0.5	0.25	0.096	2.5×10^{-5}	9.4×10^{-12}	conv. (3)
	paper	200	25	13	0.5	0.25	0.1	3×10^{-5}	7×10^{-12}	
case69limit	ours	0.012	2.4×10^{-4}	1.2×10^{-4}	1.6×10^{-6}	4.6×10^{-7}	3.9×10^{-12}	–	–	conv. (1)
	paper	0.012	2×10^{-4}	10^{-4}	2×10^{-6}	5×10^{-7}	5×10^{-12}	–	–	
case141limit	ours	6.4×10^{-3}	1.6×10^{-4}	8.2×10^{-5}	10^{-6}	4.5×10^{-7}	1.2×10^{-11}	–	–	conv. (1)
	paper	6×10^{-3}	2×10^{-4}	8×10^{-5}	10^{-6}	5×10^{-7}	6×10^{-11}	–	–	
case500limit	ours	13	1.2	0.62	0.017	8.7×10^{-3}	1.1×10^{-6}	2.6×10^{-13}	–	conv. (2)
	paper	13	1.2	0.6	0.02	9×10^{-3}	10^{-6}	2×10^{-13}	–	
case2000limit	ours	55	3.9	2	2.8	1.5	0.01	7.5×10^{-7}	5.9×10^{-13}	conv. (3)
	paper	54	3.9	2	2.8	1.5	0.01	8×10^{-7}	9×10^{-13}	

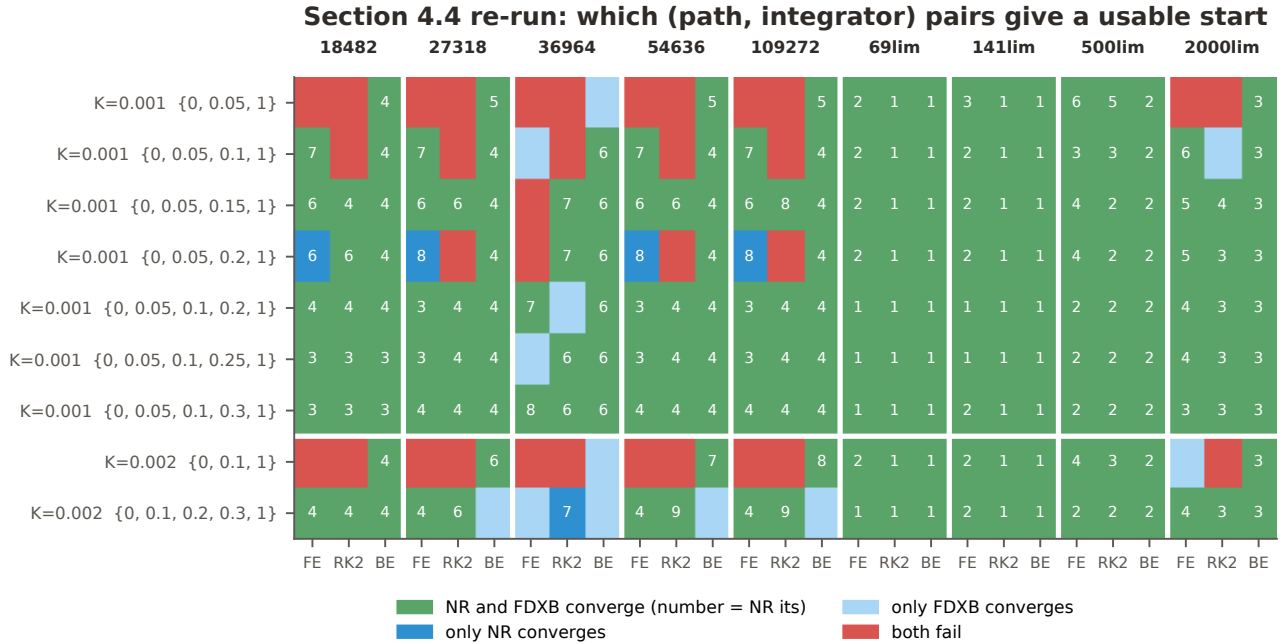


Figure 9: Section 4.4 re-run. Rows: paths and K ; columns: system and scheme. Colour: whether NR and FDXB converge afterwards; number: NR iterations.

ARPACK to four digits on 33 of the 34 systems; on case571limit it returns 0.2467 against ARPACK's 0.2273. The power method is less reliable: its error exceeds 5% on four systems, case18482 (13%), case36964 (8%), case10595 (19%) and case12110 (13%), all of which contain the 9241-bus PEGASE network, and it is 1.9% and 2.9% off on case_ieee30limit and case9241pegaselimit. The power method converges slowly when the two largest eigenvalue moduli are close; we did not measure that gap, so this is the likely, not a proven, cause.

Feasibility maps. The method works in a band along the diagonals $\delta = \text{const}$ (Figure 17), which confirms that δ , not K or Δt_0 alone, is the parameter that matters. The band's width depends on the system: almost the whole grid works for case18482 and case2000limit, but only 29 of 88 cells for case36964, with no cell at $\Delta t_0 = 0.1$ or 0.2 , which explains the paper's finding that $\Delta t_0 = 0.1$ fails although δ is unchanged. For case6024 the band lies at δ from 0.03 to 32, and the paper's $\delta = 0.02$ runs along its edge.

Table 5: Table 7 re-run: iterations of the refining solver (ours / paper). NR-MAT starts from the case file, all others from the flat start. BE uses the path $\{0, 0.005, 1\}$, RK2 the paper’s path $\{0, 0.005, 0.01, 0.02, 0.12, 0.42, 0.72, 1\}$. Mismatches with the paper are in bold.

Case	NR-MAT	NR-flat	GSH-NR	BE(NR)	RK2(NR)	BE(FDXB)	RK2(FDXB)
case18482	6 / 6	fail / fail	10 / 10	4 / 4	3 / 3	33 / 33	29 / 29
case27318	5 / 5	fail / fail	fail / 25	4 / 4	3 / 3	29 / 29	25 / 25
case36964	6 / 6	fail / fail	28 / 27	4 / 4	3 / 3	33 / 33	29 / 29
case54636	5 / 5	fail / fail	fail / fail	4 / 4	3 / 3	29 / 29	25 / 25
case109272	5 / 5	fail / fail	fail / fail	4 / 4	3 / 3	29 / 19	25 / 25
case69limit	2 / 4	2 / 4	2 / 4	1 / 1	1 / 1	8 / 8	8 / 8
case141limit	2 / 3	2 / 3	2 / 3	1 / 1	1 / 1	8 / 8	6 / 6
case500limit	3 / 3	4 / 4	4 / 4	2 / 2	2 / 2	13 / 13	11 / 11
case2000limit	3 / 3	5 / 5	5 / 5	3 / 3	3 / 3	22 / 22	20 / 20

Table 6: Eight places where the base paper is inconsistent or under-specified, the evidence, and our handling.

Issue	Evidence	Our handling
Base MVA of case69limit, case141limit BE(FDXB) = 19 for case109272 (Table 7)	Files store 10 MVA. Tables 2–5 match only on 100 MVA; Table 7’s NR-MAT/NR-flat counts (4, 3) match only on 10 MVA, where BE(NR) then needs 2, not 1. No single base reproduces both. We get 29, the same as case54636. The two systems are 8 and 4 copies of one network and are identical in every other cell of every table, the paper’s included.	Use 100 MVA; report 10 MVA counts separately. Reported; most likely a typo.
Table 2, FE on case18482 at $t = 0.01$ Fig. 1(a), $\epsilon = 0.05$	Paper 9.7, ours 3.7; every other cell of that column matches to two digits.	Reported, unresolved.
GSH-NR specification	One NR step from (1, 0.95) gives $x = (6.01, -4.01)$ and $\ g\ _\infty \approx 99.5$, i.e. $\log_{10} \approx 2$; the paper’s curve starts near 3. We need 8 iterations, not 9. Described in words only; no formula for the shunts or for δ . Our reconstruction matches case18482 (10) and nearly case36964 (28 vs 27) but fails on case27318 (paper: 25).	Computed by hand and in code.
Section 4.4, $\Delta t_0 = 0.1$	“Does not work” is stated without the path used. With $\{0, 0.1, 1\}$ BE works on 8 of 9 systems; with $\{0, 0.1, 0.2, 0.3, 1\}$ it fails on the four largest with NR.	Our version documented in gsh.py.
Table 6, RK2, $t_3 = 0.20$, case36964 Flat-start mismatch of case2000limit	The paper reports success (232%); in our run RK2 diverges on this path. Tables 2–4 print 54, Table 5 prints 55 for the same starting point (ours: 54.95).	Both paths reported (Figure 9). Reported. Cosmetic; rounding.

Table 7: Runs that reach the reference solution (of 34 per setting, 170 in total). *Other*: runs that converge to a different root. *Gained/lost* are relative to the paper’s method, with gains split into rescues / root fixes / speed-ups. *LUs S1*: mean factorizations in S1 on the systems both this configuration and the paper’s method solve (the paper’s own mean on those systems in grey where it is not 8.04). *Ratio*: the same, all settings pooled, relative to the paper’s method.

Configuration	S1	S2	S3	S4	S5	Total	Other	Gained	Lost	LUs S1	Ratio
paper	28	27	25	18	29	127	14	–	–	8.04	1.00
PC	32	32	32	22	34	152	2	+25 (2/13/10)	–0	8.43	1.04
OM + PC	32	32	32	22	34	152	3	+25 (2/13/10)	–0	7.57	0.93
OM + PC + adaptive	32	32	32	22	34	152	2	+25 (2/13/10)	–0	7.86	1.25
PC + adaptive	32	32	32	22	34	152	1	+25 (2/13/10)	–0	8.71	1.35
OM	30	30	29	20	34	143	12	+16 (1/5/10)	–0	7.14	0.88
OM + adaptive	32	33	30	20	33	148	10	+22 (4/8/10)	–1	7.46	0.92
adaptive	30	30	26	18	28	132	14	+6 (3/3/0)	–1	8.36	1.03
richardson (BE)	32	34	28	23	34	151	9	+25 (9/7/9)	–1	10.61	1.30
OM + richardson	32	33	34	23	34	156	8	+29 (10/9/10)	–0	9.75	1.18
richardson + FE	34	33	29	27	34	157	3	+31 (9/13/9)	–1	37.14	3.82
richardson + RK2	34	33	30	27	34	158	4	+32 (9/13/10)	–1	47.50	4.78
richardson + RK4	32	30	27	24	32	145	10	+27 (4/13/10)	–9	87.15 (7.96)	8.14
scaled homotopy	28	27	24	18	32	129	6	+3 (0/3/0)	–1	8.04	1.00
BE-chord	19	18	17	14	21	89	24	+9 (0/9/0)	–47	8.89 (8.17)	1.09
newton homotopy	15	15	15	15	15	75	10	+4 (0/4/0)	–56	9.00 (8.13)	1.10
RK4 (fixed steps)	2	3	4	2	2	13	1	+0 (0/0/0)	–114	7.00 (3.00)	2.36

Tuning δ . For case27318, case54636 and case109272 golden-section search lands at $\delta = 0.025$, close to the paper’s 0.02. On case36964 it lands at 0.096, and on the other eight ill-conditioned systems at 1.0 to 4.8, fifty or more times the paper’s value. On 12 of

the 22 limit cases it runs to the lower end of the interval ($\delta \approx 10^{-4}$), because there δ hardly changes the LU count (Figure 18). Rules fitted on the paper’s nine systems and tested on the other 25 (Table 8) show that a fixed $\delta = 0.011$ behaves like the paper’s 0.02, a spectral

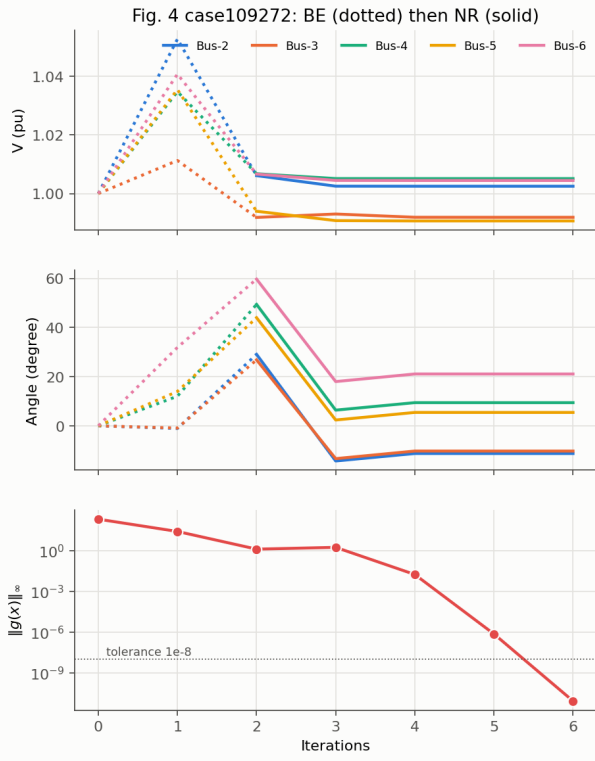


Figure 10: Figure 4 re-run: voltage magnitudes and angles of buses 2–6 of case109272 along the two-point BE path (dotted) and NR (solid). The large angle excursion at $t = 1$ is harmless: the state is already inside NR’s region of convergence.

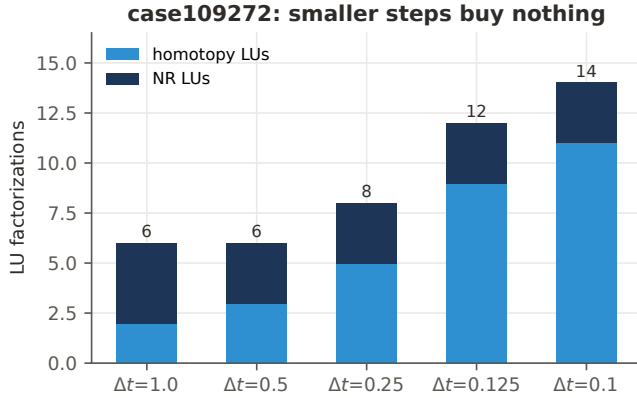


Figure 11: Figure 5 re-run in cost terms: LU factorizations for each constant step Δt after t_1 . Every run ends at the same state.

rule $\delta = c |\lambda|_{\min}$ fails (it loses four of nine training systems; the tuned δ is 54 to 819 times $|\lambda|_{\min}$), and a per-system search solves four more test systems but costs 21 solves, far more than it saves.

Own LU. On the tutorial and the limit cases of 69 to 2,000 buses our dense LU gives exactly the same iteration counts as SuperLU

Table 8: Choosing δ without a search: rules fitted on the paper’s nine systems, tested on the other 25. Mean LUs are over solved systems and exclude the search itself.

How δ is chosen	Train	LUs	Test	LUs
paper, $\delta = 0.02$	9/9	5.00	19/25	9.47
fixed, $\delta = 0.011$	9/9	5.11	19/25	9.53
spectral, $\delta = 1.86 \lambda _{\min}$	5/9	5.20	20/25	11.00
golden search per system (21 solves)	9/9	4.89	23/25	9.13

and the same solution to 5×10^{-12} (Table 9). The time shows why the paper’s systems need a sparse solver: NR on case20001 limit takes 424 s with our LU against 0.04 s with SuperLU, and the dense Jacobian of case109272 alone would need about 276 GB. Reusing factors also matters: for 20 right-hand sides with one 400×400 matrix, one LU plus 20 pairs of triangular solves is 14 times faster than 20 Gauss eliminations, which is why FDXB factorises B' and B'' once.

Numerical studies (Q5). The shift δ matters exactly where the spectrum says it should: on systems whose flat-start Jacobian has eigenvalues below δ . There the method works in a band of constant δ whose position depends on the system, and neither a fixed value nor a multiple of $|\lambda|_{\min}$ predicts the best δ . This is why the corrector, which makes more settings work instead of looking for a better one, is worth more than tuning δ .

4.5 Discussion: why the failures fail

Explicit schemes. FE at $t = 0$ steps by $-(\Delta t_0/K) g(x^{(0)})$, blind to the network, and later explicit steps to $t = 1$ still use G_x at t_1 , dominated by KI wherever J is small. The ODE is stiff in the sense of [7]: its fast modes have time scale $K/|\lambda(J)|$, which with $|\lambda|$ from 4×10^{-4} to 5×10^4 (Figure 16) spans eight orders of magnitude. An explicit method is stable only for steps of the order of the fastest of these time scales, while an implicit one is not restricted in this way [7]. RK4 is explicit, which is consistent with its failure with fixed steps (13 of 170 runs). Under Richardson control it solves 145 runs but needs 87 LUs per system in S1 against 37 for FE and 10.6 for BE: every attempt costs twelve LUs (three steps of four stages), and its higher order does not buy larger steps. Our reading is that the steps are limited by stability rather than accuracy; we did not measure the accepted RK4 step sizes.

BE-chord. Its fixed-point iteration has derivative

$$-\Delta t G_x(x_k, t_{k+1})^{-1} (J(\hat{x}) - KI).$$

On the last step $G_x = J(x_k)$, so when $J(\hat{x}) \approx J(x_k)$ the iteration matrix is close to $-\Delta t I$ with $\Delta t = 0.995$: its spectral radius is about one, so the iteration does not contract. The first iterate is the BE step itself; the second and third can only move away from it. BE-chord loses 47 runs and lands on another root 24 times. This is our analysis of why it fails; we measured the outcome, not the spectral radius.

Newton and scaled homotopies. The Newton homotopy has $G_x = J(x)$ at every t and no KI term; its BE first step is $-\Delta t_0 J(x^{(0)})^{-1} g(x^{(0)})$, a plain Newton step shrunk to Δt_0 (0.5% in S1) with the flat-start Jacobian unregularised. It solves only 75

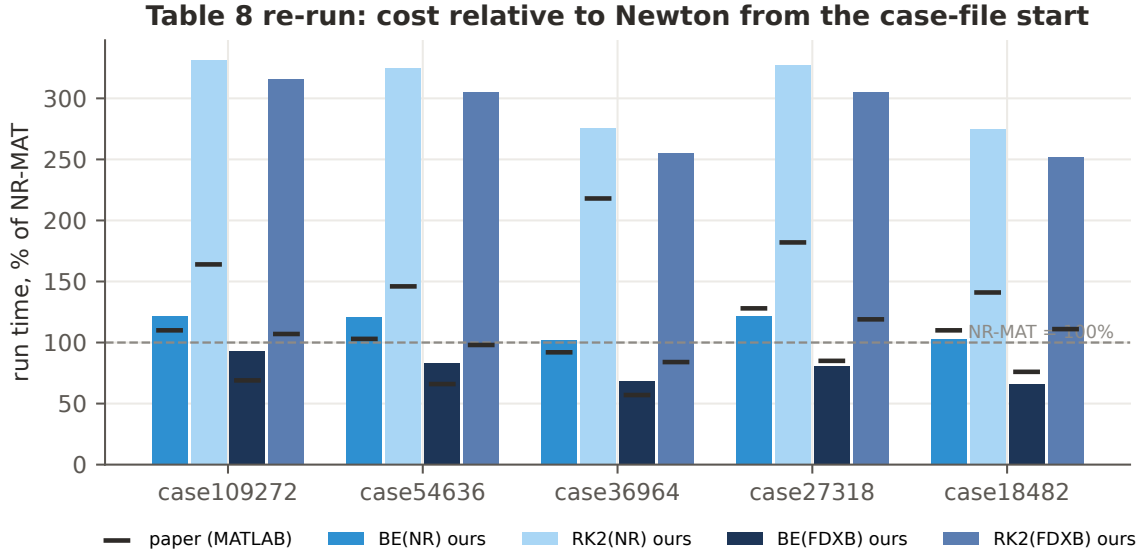


Figure 12: Table 8 re-run: median run time of seven repetitions as a percentage of NR-MAT (bars) against the paper's MATLAB percentages (black ticks).

Table 9: The paper's method (BE path $\{0, 0.005, 1\}$, $K = 10^{-4}$) with SciPy's sparse LU and with our dense LU with partial pivoting: refiner iterations, largest difference in the solution, and run time.

System	Refiner	Its SuperLU	Its ours	$\max \Delta x $	SuperLU (s)	Ours (s)
tutorial	NR	9	9	0	0.002	0.002
case69limit	NR	1	1	3.6×10^{-13}	0.004	0.011
	FDXB	8	8	1.6×10^{-13}	0.004	0.010
case141limit	NR	1	1	9.7×10^{-14}	0.005	0.039
	FDXB	8	8	4.7×10^{-12}	0.004	0.032
case500limit	NR	2	2	9.8×10^{-15}	0.010	3.45
	FDXB	13	13	1.2×10^{-14}	0.008	1.71
case2000limit	NR	3	3	4.3×10^{-14}	0.040	424.1
	FDXB	22	22	6.2×10^{-14}	0.029	100.8

of 170 runs; in S1 it fails on every ill-conditioned system except case27318 and case54636, and on case109272 it reaches another root. This is the clearest evidence in the project that the useful ingredient is the KI shift, not continuation as such. The scaled homotopy solves 129 runs against the paper's 127: the same as the paper in S1, S2 and S4, one fewer in S3 (case27318) and three more in S5, all three root fixes. Since D is normalised to mean one, equations with a small Jacobian diagonal get a *smaller* shift than in the paper's method; our interpretation is that this is why it does not help more.

Accounting and acceptance tests. At $t = 1$ the corrector's step is exactly one NR step taken before the 10-iteration budget starts counting, so 10 of its 25 gains are budget-edge speed-ups; the other 15 (two rescues and 13 root fixes) are real. The adaptive rule accepts a step only if $\|g\|_\infty$ decreases, but on the path $g(x(t)) = -\frac{1-t}{t}K(x(t) - x^{(0)})$ need not fall monotonically, so it rejects good steps (14 LUs instead of 5 on case2000limit). Richardson control compares two integrations with each other, not with the homotopy

curve, so both can drift towards another branch (9 wrong roots with BE, against 2 for the corrector).

Strengths of this study. Every number in this report is produced by a script in the repository, and every reproduced value is printed next to the paper's; the building blocks are tested before any comparison; the algebra of Section 2.2 is consistent with which modifications work (those that improve the Newton stages) and which fail (the one that removes the shift); and the modification study is a controlled experiment on 34 systems with a machine-independent cost and a root check. Seven runs, all with the weak K of S4 (case36964, case6024, case6243, case6748, case9961, case3012wplimit, case3375wplimit), are solved by no configuration.

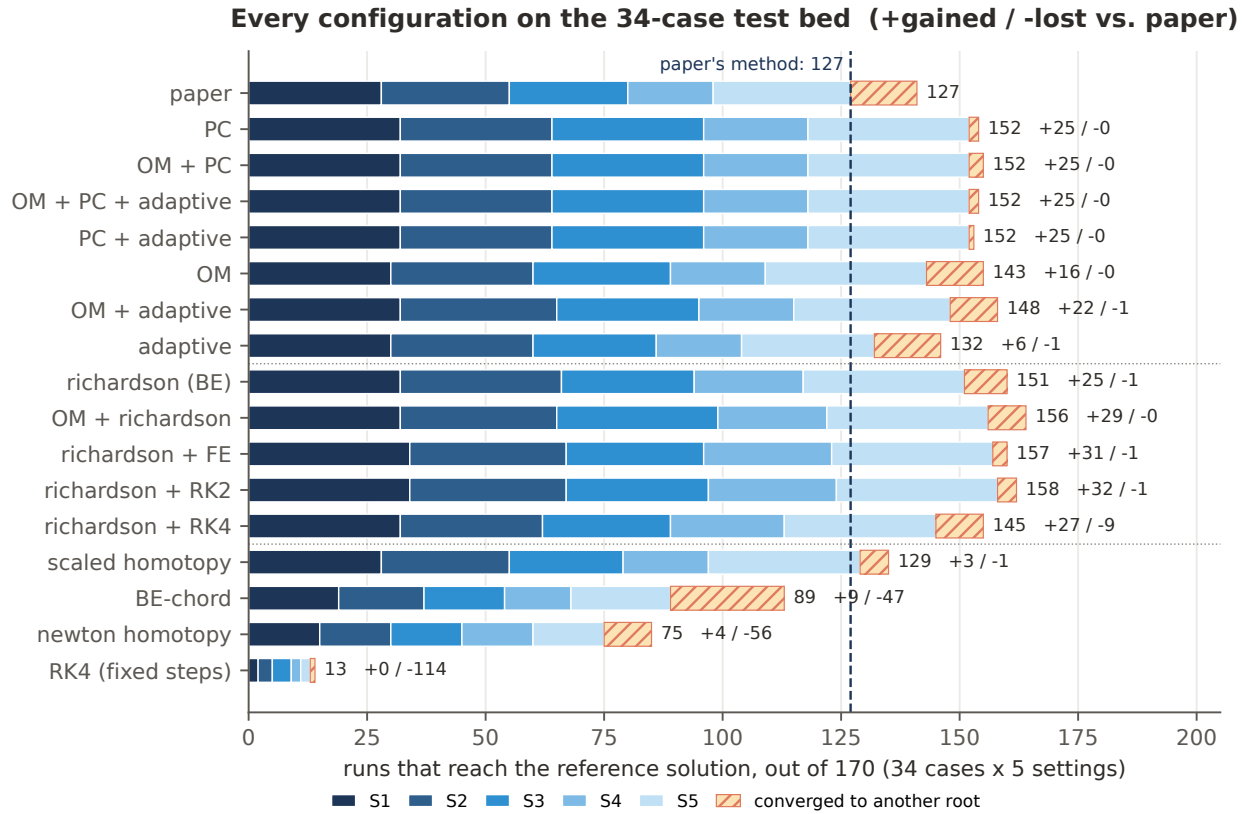


Figure 13: Runs that reach the reference solution per configuration, stacked by setting; hatched: runs that converge to another root. Dashed line: the paper's method.

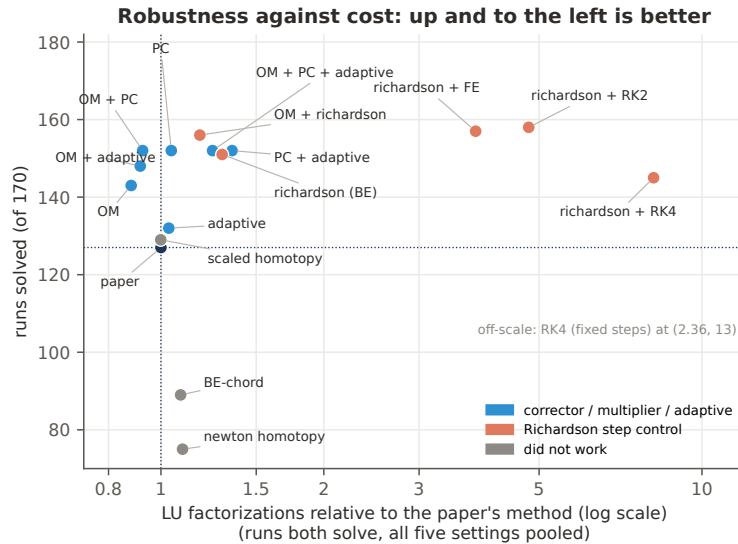


Figure 14: Runs solved against LU cost relative to the paper's method (log scale). Blue: corrector, multiplier, adaptive and combinations; orange: Richardson control; grey: the four ideas that did not work.

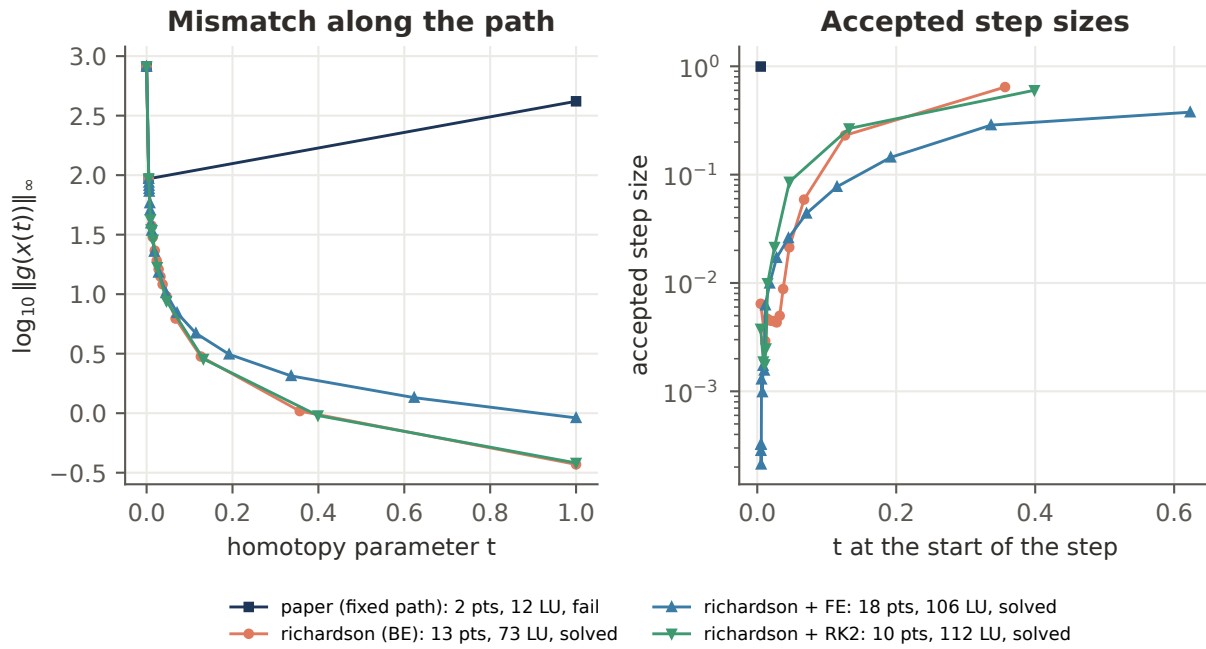


Figure 15: Richardson-controlled paths on case6024, setting S1, the one S1 system only Richardson control solves. Left: mismatch at each accepted point (legend: points, LUs including rejected attempts, and outcome). Right: accepted step sizes.

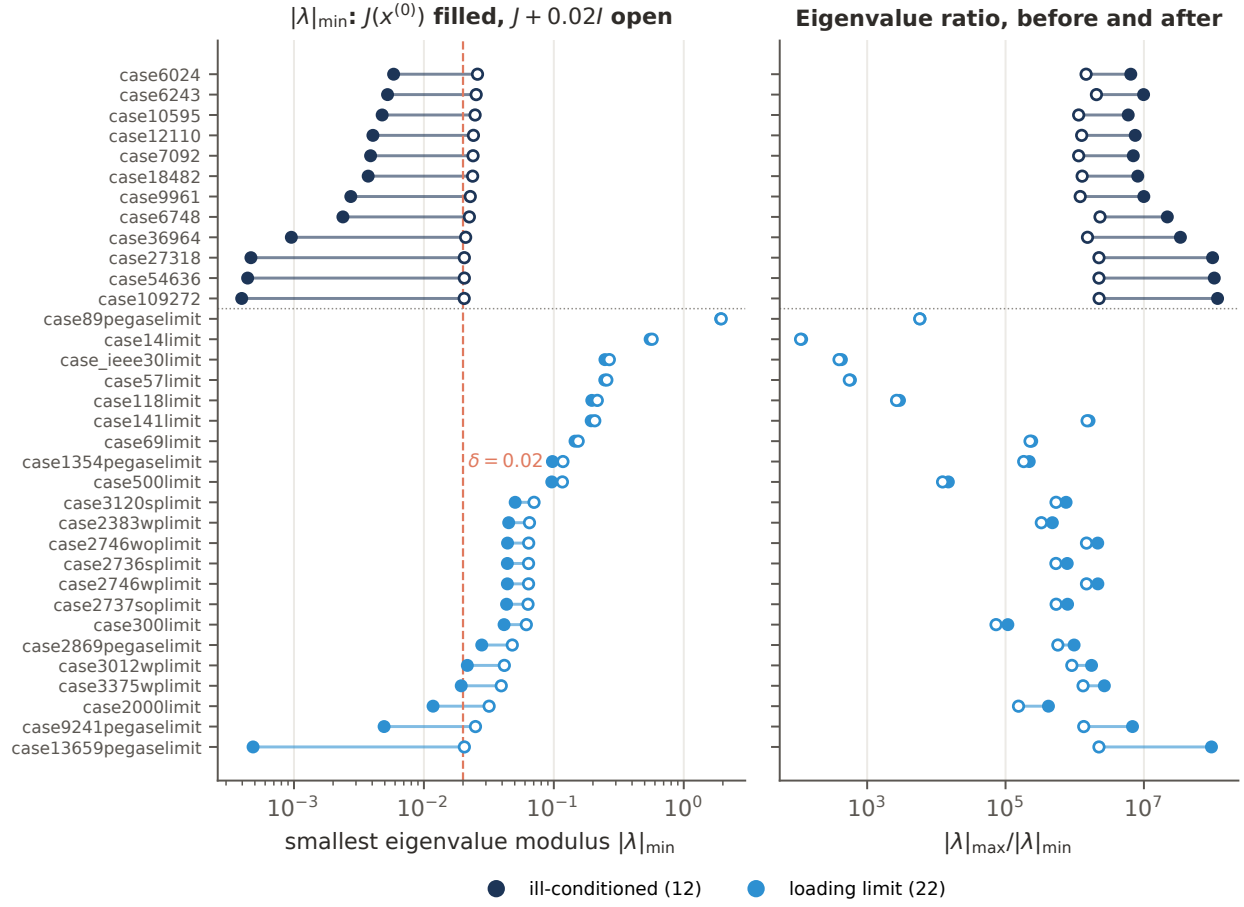


Figure 16: Smallest eigenvalue modulus (left) and $|\lambda|_{\max}/|\lambda|_{\min}$ (right) of $J(x^{(0)})$ (filled) and of $J + 0.02I$ (open) for all 34 systems. The shift matters only where $|\lambda|_{\min} < \delta$.

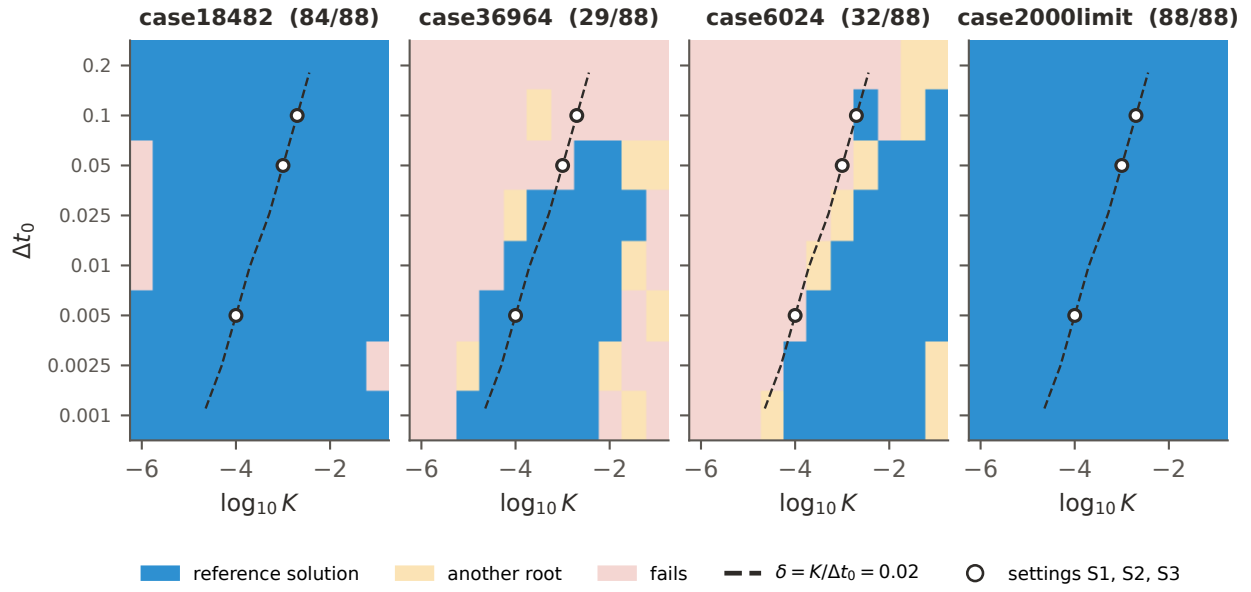


Figure 17: Outcome of the paper's method on a grid of K and Δt_0 . Dashed: $\delta = 0.02$, the paper's shift. Circles: settings S1, S2 and S3.

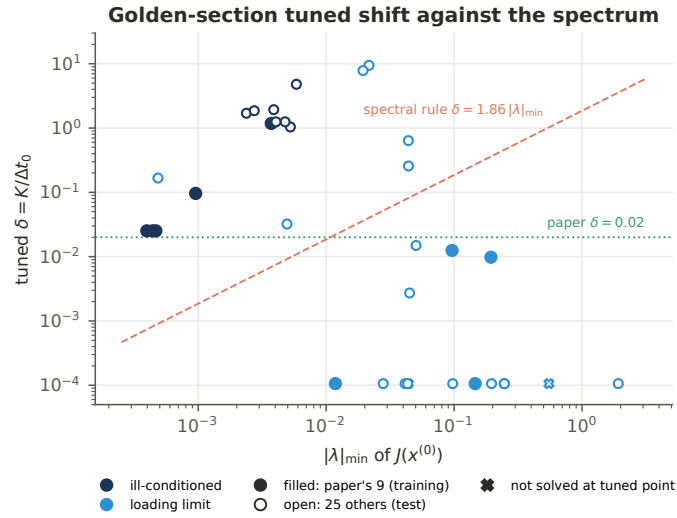


Figure 18: Golden-section tuned δ against $|\lambda|_{\min}(J(x^{(0)}))$. Points at 10^{-4} : the search ran to the lower end because δ does not matter. A spectral rule would put the points on a line of slope one.

5 Conclusion and Future Work

Key takeaways. The base paper reproduces: on all five ill-conditioned grids of up to 109,272 buses NR from the flat start fails, and a two-point backward-Euler homotopy turns that into a four-iteration solve, with our numbers matching the paper’s across 28 orders of magnitude. The method is simpler than its derivation suggests: the first BE step is a Newton step on a Jacobian shifted by $\delta = K/\Delta t_0$ and the second is almost a plain Newton step; BE wins because it is the only scheme that uses the real Jacobian where it matters. On a test bed four times the paper’s, the most important question turned out to be *which solution* a converged run reaches: the paper’s method converges on 141 of 170 runs but reaches the intended operating point on 127. A Newton corrector raises that to 152 and cuts wrong-root runs from 14 to 2; with Iwamoto’s optimal multiplier it does so with 7% fewer LU factorizations, and this is the configuration we recommend. Richardson control solves systems nothing else solves at 18–30% more cost and confirms under fair conditions that BE is the right integrator. The shift matters where the flat-start Jacobian has eigenvalues below it, and no fixed or spectral rule predicts its best value.

Limitations. The test bed is two Zenodo records, and the ill-conditioned systems are replicas of a few base networks, so they carry less independent evidence than unrelated grids would. The

reference operating point is a convention (the root NR reaches from the stored guess), and we did not study whether the other roots are physically meaningful. The modification study used only paths $\{0, \Delta t_0, 1\}$; the Richardson tolerance and golden-section brackets were set once and not tuned; the feasibility maps cover four systems; our GSH-NR is a reconstruction from a verbal description; and timings are noisy Python measurements on a laptop.

Future work. (i) A Richardson path with a corrector after each accepted point, to combine the most rescues with the fewest wrong roots. (ii) Cheaper error estimates, through an embedded pair or one LU reused for the three steps of each attempt, which would cut Richardson’s cost by up to three times. (iii) A path-aware acceptance test for adaptive steps based on $\|G(x, t)\|$ instead of $\|g(x)\|$. (iv) A scaling that raises the shift on weakly coupled equations. (v) A sparse LU from scratch with a fill-reducing ordering, to run the whole pipeline on our own linear algebra.

Acknowledgments

We thank our course teacher, Anup Halder Joy (Department of CSE, BUET), for guidance throughout the CSE 402 Numerical Analysis Sessional, and the authors of the base paper and of the Zenodo test-system records for making their work public.

References

- [1] Eugene L. Allgower and Kurt Georg. 2003. *Introduction to Numerical Continuation Methods*. Classics in Applied Mathematics, Vol. 45. SIAM, Philadelphia, PA. doi:10.1137/1.9780898719154
- [2] Stéphane Fliscounakis, Patrick Panciatici, Florin Capitanescu, and Louis Wehenkel. 2013. Contingency Ranking with Respect to Overloads in Very Large Power Systems Taking into Account Uncertainty, Preventive, and Corrective Actions. *IEEE Transactions on Power Systems* 28, 4 (2013), 4909–4917.
- [3] Francisco Damasceno Freitas and Leandro Nunes de Oliveira. 2023. Conditioning Step on the Initial Estimate When Solving Ill-Conditioned Power Flow Problems. *International Journal of Electrical Power & Energy Systems* 146 (2023), 108772.
- [4] Francisco Damasceno Freitas and Alisson Lima Silva. 2022. Flat Start Guess Homotopy-Based Power Flow Method Guided by Fictitious Network Compensation Control. *International Journal of Electrical Power & Energy Systems* 142 (2022), 108311.
- [5] Gene H. Golub and Charles F. Van Loan. 2013. *Matrix Computations* (4th ed.). Johns Hopkins University Press, Baltimore, MD.
- [6] Ernst Hairer, Syvert P. Nørsett, and Gerhard Wanner. 1993. *Solving Ordinary Differential Equations I: Nonstiff Problems* (2nd ed.). Springer, Berlin. doi:10.1007/978-3-540-78862-1
- [7] Ernst Hairer and Gerhard Wanner. 1996. *Solving Ordinary Differential Equations II: Stiff and Differential-Algebraic Problems* (2nd ed.). Springer, Berlin. doi:10.1007/978-3-642-05221-7
- [8] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, et al. 2020. Array Programming with NumPy. *Nature* 585 (2020), 357–362. doi:10.1038/s41586-020-2649-2
- [9] John D. Hunter. 2007. Matplotlib: A 2D Graphics Environment. *Computing in Science & Engineering* 9, 3 (2007), 90–95. doi:10.1109/MCSE.2007.55
- [10] S. Iwamoto and Y. Tamura. 1981. A Load Flow Calculation Method for Ill-Conditioned Power Systems. *IEEE Transactions on Power Apparatus and Systems* PAS-100, 4 (1981), 1736–1743.
- [11] J. Kiefer. 1953. Sequential Minimax Search for a Maximum. *Proc. Amer. Math. Soc.* 4, 3 (1953), 502–506. doi:10.1090/S0002-9939-1953-0055639-3
- [12] Holger Krekel et al. 2004. *pytest*. <https://github.com/pytest-dev/pytest>
- [13] Richard B. Lehoucq, Danny C. Sorensen, and Chao Yang. 1998. *ARPACK Users’ Guide: Solution of Large-Scale Eigenvalue Problems with Implicitly Restarted Arnoldi Methods*. SIAM, Philadelphia, PA. doi:10.1137/1.9780898719628
- [14] Kenneth Levenberg. 1944. A Method for the Solution of Certain Non-Linear Problems in Least Squares. *Quart. Appl. Math.* 2, 2 (1944), 164–168. doi:10.1090/qam/10666
- [15] Xiaoye S. Li. 2005. An Overview of SuperLU: Algorithms, Implementation, and User Interface. *ACM Trans. Math. Software* 31, 3 (2005), 302–325. doi:10.1145/1089014.1089017
- [16] Alisson Lima-Silva and Francisco Damasceno Freitas. 2024. Dynamical Homotopy Transient-Based Technique to Improve the Convergence of Ill-Posed Power Flow

- Problem. *International Journal of Electrical Power & Energy Systems* 155 (2024), 109436.
- [17] Alisson Lima-Silva and Francisco Damasceno Freitas. 2024. Exploring a Dynamic Homotopy Technique to Enhance the Convergence of Classical Power Flow Iterative Solvers in Ill-Conditioned Power System Models. *Energies* 17, 18 (2024), 4642. doi:10.3390/en17184642
 - [18] Richard Lincoln. 2015. *PYPOWER: A Power Flow and Optimal Power Flow Solver, Port of MATPOWER to Python*. <https://github.com/rwl/PYPOWER>
 - [19] Donald W. Marquardt. 1963. An Algorithm for Least-Squares Estimation of Nonlinear Parameters. *J. Soc. Indust. Appl. Math.* 11, 2 (1963), 431–441. doi:10.1137/0111030
 - [20] Federico Milano. 2009. Continuous Newton’s Method for Power Flow Analysis. *IEEE Transactions on Power Systems* 24, 1 (2009), 50–57.
 - [21] Mohiuddin Sizan, Sadman bin Tareq, Mosharaf Hossain Apurbo, Elias Hasbi, and Jonayed Mohiuddin Shanto. 2026. *DynHomotopy: Dynamic Homotopy for Ill-Conditioned Power Flow, Source Code*. <https://github.com/JonayedMohiuddin/NumericalAnalysisProject>
 - [22] B. Stott and O. Alsac. 1974. Fast Decoupled Load Flow. *IEEE Transactions on Power Apparatus and Systems* PAS-93, 3 (1974), 859–869.
 - [23] William F. Tinney and Clifford E. Hart. 1967. Power Flow Solution by Newton’s Method. *IEEE Transactions on Power Apparatus and Systems* PAS-86, 11 (1967), 1449–1460.
 - [24] Marcos Tostado-Véliz, Salah Kamel, and Francisco Jurado. 2019. Matpower Ill-Conditioned Systems. Data set, Zenodo, record 3514739. <https://zenodo.org/records/3514739>
 - [25] Marcos Tostado-Véliz, Salah Kamel, and Francisco Jurado. 2019. Matpower Limit Cases. Data set, Zenodo, record 3491654. <https://zenodo.org/records/3491654>
 - [26] Robert A. M. van Amerongen. 1989. A General-Purpose Version of the Fast Decoupled Load Flow. *IEEE Transactions on Power Systems* 4, 2 (1989), 760–770.
 - [27] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, et al. 2020. SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods* 17 (2020), 261–272. doi:10.1038/s41592-019-0686-2
 - [28] Layne T. Watson. 1989. Globally Convergent Homotopy Methods: A Tutorial. *Appl. Math. Comput.* 31 (1989), 369–396.
 - [29] Ray D. Zimmerman, Carlos E. Murillo-Sánchez, and Robert J. Thomas. 2011. MATPOWER: Steady-State Operations, Planning, and Analysis Tools for Power Systems Research and Education. *IEEE Transactions on Power Systems* 26, 1 (2011), 12–19. doi:10.1109/TPWRS.2010.2051168

A Reproducing This Report

All results come from the public repository [21] (<https://github.com/JonayedMohiuddin/NumericalAnalysisProject>). Test systems are downloaded from Zenodo on first use. Experiments 1–10 (the paper) take about an hour on a laptop; experiment 11 (17 configurations $\times$ 170 runs) takes several hours.

```
pip install -r requirements.txt
python -m pytest tests                # 24 tests
python run_all.py                     # every experiment (1-15)
python run_all.py --only 3 4          # selected experiments
python -m improvements case36964 --corrector --multiplier
python -m improvements case6024 --richardson --dt0 0.05 --K 1e-3
python -m improvements spectrum case18482
python -m improvements feasibility case6024
python -m improvements tune case6024
python -m improvements scratch case_ACTIVSg500limit --refiner FDXB
python report/scripts/figures_from_results.py
python report/scripts/figures_from_code.py
```

Table 10: Map from each result to the script that produces it and its output in results/.

Result	Script	Output
Figures 1 and 2	exp01_tutorial.py	fig1*.png, fig2*.png
Section 4.2.1	exp02_first_step.py	sec421_first_step.md
Tables 2, 3, 4	exp03_tables2to4.py	table2_FE.md, table3_RK2.md, table4_BE.md
Table 5	exp04_table5.py	table5.md
Section 4.4	exp05_sec44_pathways.py	sec44_pathways.md
Figure 3, Table 6	exp06_fig3_table6.py	fig3*.png, table6.md
Figure 4	exp07_fig4.py	fig4_case109272_states.png
Figure 5	exp08_fig5.py	fig5_timestep_sensitivity.png
Table 7	exp09_table7.py	table7.md, table7_native_base.md
Table 8	exp10_table8.py	table8.md
Modifications	exp11_improvements.py	improvements_*.md
Spectrum	exp12_spectrum.py	spectrum.md
($K, \Delta t_0$) maps	exp13_feasibility.py	feasibility.md
Tuning δ	exp14_tuning.py	tuning_per_case.md, tuning_rules.md
Own LU	exp15_scratch_lu.py	scratch_lu.md, gauss_vs_lu.md

B Full Tables

Table 11: Table 2 re-run: FE after a BE first step, path $\{0, 0.005, 0.01, 0.51, 1\}$, $K = 10^{-4}$ (our value above the paper’s).

Case		$t=0$	0.005	0.01	0.51	1	NR 1	NR 2	NR 3	Result
case18482	ours	533	65	3.7	1943	1.3×10^5	2.6×10^6	1.4×10^7	8.9×10^9	no
	paper	532	65	9.7	2000	2000	–	–	–	
case27318	ours	201	25	1.2	27	1.6×10^7	3.3×10^7	7.5×10^7	1.7×10^8	no
	paper	200	25	1.2	27	2×10^7	–	–	–	
case36964	ours	533	66	3.5	1837	2.7×10^5	7.7×10^9	7.7×10^{11}	7×10^{12}	no
	paper	532	65	3.5	2000	3×10^5	–	–	–	
case54636	ours	201	25	1.2	27	1.1×10^9	9.6×10^9	7.1×10^9	1.6×10^{10}	no
	paper	200	25	1.2	27	10^9	–	–	–	
case109272	ours	201	25	1.2	27	1.1×10^9	9.6×10^9	7.1×10^9	1.6×10^{10}	no
	paper	200	25	1.2	27	10^9	–	–	–	
case69limit	ours	0.012	2.4×10^{-4}	1.2×10^{-5}	5.4×10^{-4}	2.4×10^{-5}	1.7×10^{-8}	4.3×10^{-12}	–	conv. (2)
	paper	0.012	2×10^{-4}	10^{-5}	5×10^{-4}	2×10^{-5}	2×10^{-8}	3×10^{-12}	–	
case141limit	ours	6.4×10^{-3}	1.6×10^{-4}	8.6×10^{-6}	4.6×10^{-4}	2.3×10^{-5}	3.9×10^{-8}	1.7×10^{-11}	–	conv. (2)
	paper	6×10^{-3}	2×10^{-4}	9×10^{-6}	5×10^{-4}	2×10^{-5}	4×10^{-8}	10^{-11}	–	
case500limit	ours	13	1.2	0.022	0.92	1	8.8×10^{-3}	1.9×10^{-6}	3.9×10^{-13}	conv. (3)
	paper	13	1.2	0.02	0.9	1	8×10^{-3}	2×10^{-6}	4×10^{-13}	
case2000limit	ours	55	3.9	0.78	210	6802	3.2×10^7	5.4×10^7	1.2×10^8	no
	paper	54	3.9	0.8	210	7000	–	–	–	

Table 12: Table 3 re-run: RK2 after a BE first step, same path and K .

Case		$t=0$	0.005	0.01	0.51	1	NR 1	NR 2	NR 3	Result
case18482	ours	533	65	33	2.5×10^7	6×10^8	3.4×10^8	5.9×10^{10}	9.9×10^{12}	no
	paper	532	65	33	3×10^7	10^{11}	–	–	–	
case27318	ours	201	25	13	5.8×10^5	1.6×10^{11}	4.2×10^{10}	9.4×10^{10}	2.1×10^{11}	no
	paper	200	25	13	6×10^5	10^{10}	–	–	–	
case36964	ours	533	66	33	2.3×10^9	4.3×10^{12}	4.8×10^{12}	1.1×10^{13}	2.4×10^{15}	no
	paper	532	65	33	3×10^9	2×10^{14}	–	–	–	
case54636	ours	201	25	13	6.9×10^5	1.6×10^{12}	1.2×10^{12}	2.7×10^{12}	6×10^{12}	no
	paper	200	25	13	7×10^5	10^{10}	–	–	–	
case109272	ours	201	25	13	1.1×10^7	1.6×10^{12}	1.2×10^{12}	3×10^{14}	7.5×10^{13}	no
	paper	200	25	13	10^7	5×10^{12}	–	–	–	
case69limit	ours	0.012	2.4×10^{-4}	1.2×10^{-4}	9.3×10^{-5}	4.7×10^{-5}	1.2×10^{-9}	–	–	conv. (1)
	paper	0.012	2×10^{-4}	10^{-4}	9×10^{-5}	5×10^{-5}	10^{-9}	–	–	
case141limit	ours	6.4×10^{-3}	1.6×10^{-4}	8.2×10^{-5}	4.3×10^{-4}	2.2×10^{-4}	7×10^{-9}	–	–	conv. (1)
	paper	6×10^{-3}	2×10^{-4}	8×10^{-5}	4×10^{-4}	2×10^{-4}	7×10^{-9}	–	–	
case500limit	ours	13	1.2	0.62	1.1×10^4	2.1×10^4	4.8×10^4	1.1×10^5	2.4×10^5	no
	paper	13	1.2	0.6	10^4	5000	–	–	–	
case2000limit	ours	55	3.9	1.9	1.2×10^4	4.5×10^4	10^5	5.3×10^5	1.5×10^8	no
	paper	54	3.9	1.9	10^4	4×10^4	–	–	–	

Table 13: Table 6 re-run: median CPU time of seven runs as a percentage of NR-MAT, with NR iterations after the homotopy in brackets (ours / paper).

Method	t_3	case109272	case36964	case2000limit
NR-MAT	–	1.784 s (100%)	0.797 s (100%)	0.024 s (100%)
FE	0.20	143% [3] / 146%	185% [7] / 178%	272% [4] / 68%
FE	0.25	143% [3] / 145%	fail / fail	272% [4] / 75%
FE	0.30	164% [4] / 170%	201% [8] / 179%	239% [3] / 72%
RK2	0.20	225% [4] / 230%	fail / 232%	341% [3] / 82%
RK2	0.25	224% [4] / 226%	222% [6] / 190%	340% [3] / 84%
RK2	0.30	224% [4] / 237%	221% [6] / 195%	342% [3] / 73%
BE	0.20	163% [4] / 171%	168% [6] / 144%	238% [3] / 61%
BE	0.25	164% [4] / 171%	169% [6] / 144%	237% [3] / 81%
BE	0.30	163% [4] / 169%	170% [6] / 148%	238% [3] / 94%

C Additional Figures and Tables

This appendix collects the remaining reproduced figures and tables of the base paper and the detailed per-system results of the modification study referred to in Section 4.

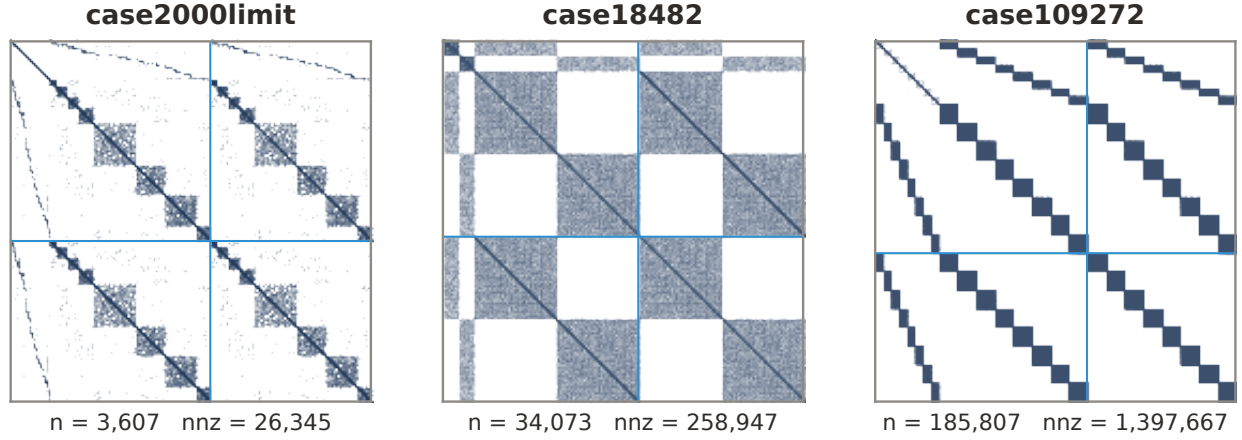


Figure 19: Nonzero pattern of the flat-start Jacobian. Orange lines separate angle from magnitude unknowns, showing the four blocks $\partial P/\partial\theta$, $\partial P/\partial V$, $\partial Q/\partial\theta$, $\partial Q/\partial V$. The ill-conditioned systems are copies of one network on the diagonal.

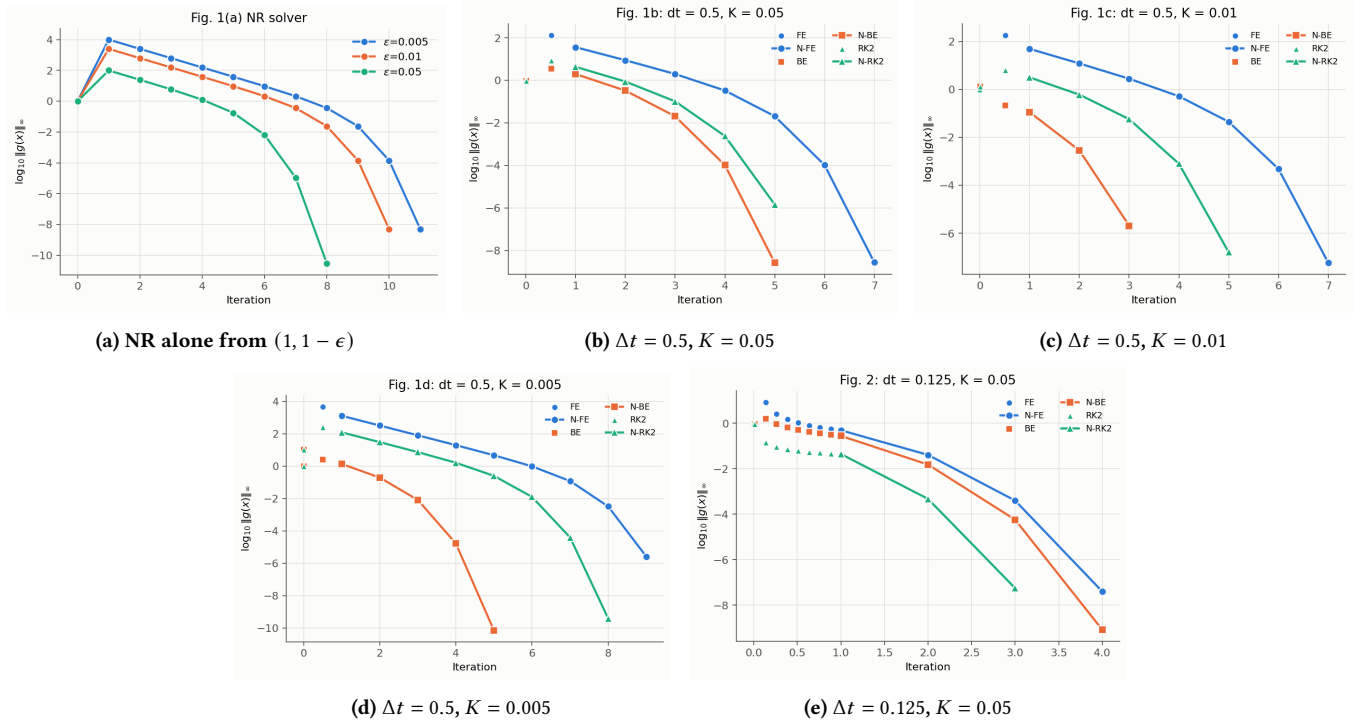
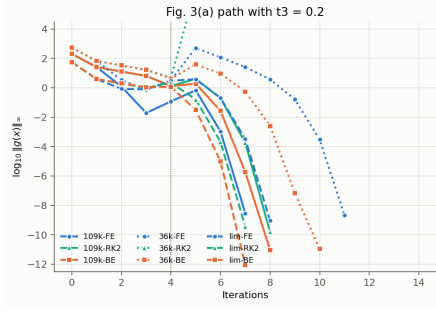
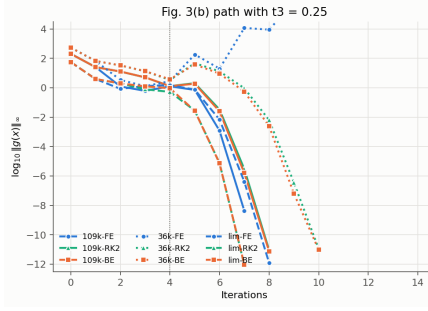
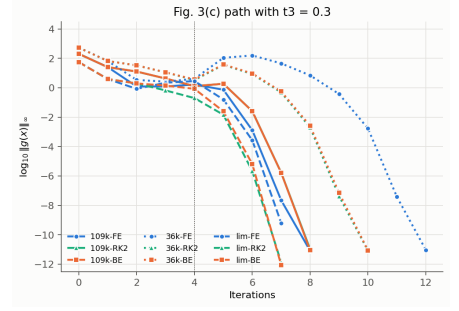
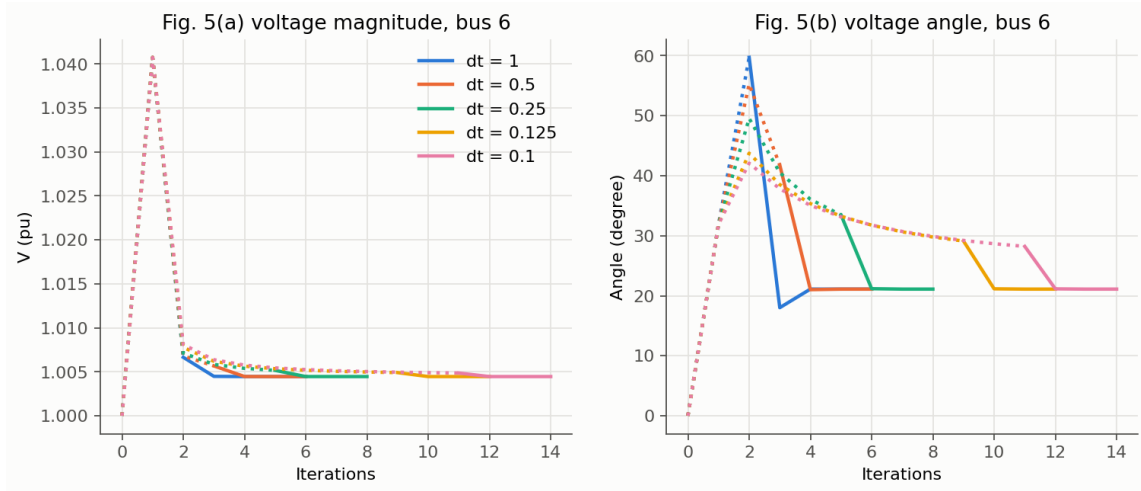


Figure 20: Our versions of the paper’s tutorial Figures 1(a)–(d) and 2. Scattered markers are homotopy points at their time $t \in [0, 1]$; connected curves are the NR iterations that follow, iteration i drawn at $1 + i$. BE (orange squares) reaches 10^{-5} first in every hybrid panel; in (c) and (d) FE and RK2 converge, but to root B .

Table 14: Table 5 re-run, $K = 10^{-4}$: $\|g\|_\infty$ at each point of the growing-step path and after three NR iterations.

Case	Solver		$t=0$	0.005	0.01	0.02	0.12	0.62	1	NR 1	NR 2	NR 3
case36964	FE	ours	533	66	3.5	0.72	5.7	57	114	28	4.7	0.23
		paper	532	65	3.5	0.7	5.7	57	114	28	5	0.2
	RK2	ours	533	66	33	16	9.6	4.2	2.6	0.19	3×10^{-4}	9.8×10^{-10}
		paper	532	65	33	16	9.6	4.2	2.6	0.2	3×10^{-4}	10^{-9}
	BE	ours	533	66	33	17	4	1.4	0.91	0.091	8.7×10^{-5}	10^{-10}
		paper	532	65	33	16	4	1.4	0.9	0.09	9×10^{-5}	10^{-10}
case109272	FE	ours	201	25	1.2	0.019	0.074	0.29	0.17	0.03	6.3×10^{-4}	2.1×10^{-9}
		paper	200	25	1.2	0.02	0.07	0.3	0.17	0.03	6×10^{-4}	2×10^{-9}
	RK2	ours	201	25	13	6.3	1.2	0.51	0.31	0.19	1.6×10^{-4}	2.2×10^{-9}
		paper	200	25	13	6.3	1.2	0.5	0.3	0.19	2×10^{-4}	2×10^{-9}
	BE	ours	201	25	13	6.3	1.1	0.21	0.13	0.017	10^{-6}	9.5×10^{-12}
		paper	200	25	13	6.3	1.1	0.2	0.1	0.02	10^{-6}	10^{-11}
case2000limit	FE	ours	55	3.9	0.78	0.66	3.6	58	91	16	1	6.2×10^{-3}
		paper	55	3.9	0.8	0.7	3.6	58	91	16	1	6×10^{-3}
	RK2	ours	55	3.9	1.9	0.97	5	5.5	3.4	0.21	3.5×10^{-4}	9.8×10^{-10}
		paper	55	3.9	1.9	1	5	5.5	3.4	0.2	3×10^{-4}	10^{-9}
	BE	ours	55	3.9	2	1	0.98	0.25	0.16	4.8×10^{-4}	2.3×10^{-9}	–
		paper	55	3.9	1.9	1	1	0.3	0.2	5×10^{-4}	3×10^{-9}	–

(a) $t_3 = 0.20$ (b) $t_3 = 0.25$ (c) $t_3 = 0.30$ **Figure 21: Figure 3 re-run: $\log_{10} \|g\|_\infty$ over five homotopy points (0–4) and the NR iterations that follow, for case109272 (solid), case36964 (dotted) and case2000limit (dashed).****Figure 22: Figure 5 re-run: voltage magnitude (a) and angle (b) of bus 6 of case109272 for five constant steps Δt after t_1 ; dotted is BE, solid is NR. Every run ends at 1.0045 pu and 21.10 degrees.**

		paper	PC	OM + PC	OM + PC + adaptive	PC + adaptive	OM	OM + adaptive	adaptive	richardson (BE)	OM + richardson	richardson + FE	richardson + RK2	richardson + RK4	scaled homotopy	BE-chord	newton homotopy	RK4 (fixed steps)	
case18482	6	6	6	6	6	5	5	6	7	7	43	46	88	6	o	x	x	paper's 9 cases	
case27318	6	6	6	6	6	6	6	6	7	7	29	46	88	6	o	8	x		
case36964	6	6	6	6	6	6	6	6	13	13	46	46	88	6	x	x	x		
case54636	6	6	6	6	6	6	6	6	7	7	29	46	88	6	o	8	x		
case109272	6	6	6	6	6	6	6	6	7	7	29	46	88	6	o	o	x		
case69limit	3	4	4	4	4	3	3	3	5	5	8	9	15	3	3	3	7		
case141limit	3	4	4	4	4	3	3	3	5	5	8	9	15	3	3	3	7		
case500limit	4	5	5	5	5	4	4	4	6	6	28	34	63	4	5	5	x		
case2000limit	5	6	6	6	6	5	14	14	6	6	34	51	87	5	6	6	x		
case6024	x	x	x	x	x	x	x	x	73	73	106	112	x	x	x	x	x	7 extra ill-conditioned	
case6243	6	6	6	6	6	6	6	6	7	7	43	46	88	6	x	x	x		
case6748	x	x	x	x	x	x	18	19	49	49	52	52	100	x	x	x	x		
case7092	6	6	6	6	6	6	6	6	7	7	40	46	76	6	7	x	x		
case9961	6	6	6	6	6	6	6	6	7	7	40	46	88	6	7	x	x		
case10595	6	6	6	6	6	5	5	6	7	7	46	46	88	6	o	x	x		
case12110	5	6	6	6	6	5	5	5	7	7	46	52	88	6	6	x	x		
case14limit	x	13	10	10	13	10	10	x	14	12	34	57	129	x	x	x	x	18 extra limit cases	
case_ieee30limit	10	11	9	9	11	9	9	10	11	10	32	61	102	10	11	11	x		
case57limit	11	11	9	9	11	9	9	11	12	10	31	53	139	10	11	11	x		
case89pegaselimit	11	11	9	9	11	9	9	11	12	10	32	54	141	11	12	12	x		
case118limit	10	11	9	9	11	9	9	10	11	9	43	54	o	10	11	11	x		
case300limit	10	11	9	9	11	8	8	10	11	9	37	49	78	10	11	11	x		
case1354pegaselimit	12	12	10	10	12	9	9	12	13	10	44	55	104	12	12	12	x		
case2383wplimit	10	11	10	18	19	9	9	10	11	10	40	54	78	10	11	11	x		
case2736splimit	x	13	9	9	13	11	11	x	14	12	45	51	105	x	x	x	x		
case2737soplimit	12	12	10	10	12	10	10	12	13	11	47	62	92	12	x	x	x		
case2746woplimit	12	12	10	10	12	10	10	12	13	11	47	50	92	12	12	x	x		
case2746wplimit	12	12	10	10	12	10	10	12	13	11	44	56	116	12	x	x	x		
case2869pegaselimit	10	11	10	10	11	9	9	10	11	10	42	54	90	10	11	11	x		
case3012wplimit	o	11	10	10	11	o	18	19	o	o	53	56	116	o	x	x	x		
case3120splimit	12	12	10	10	12	10	10	12	13	10	47	56	92	12	x	x	x		
case3375wplimit	o	11	9	9	11	o	o	o	o	o	53	50	91	o	12	x	x		
case9241pegaselimit	11	11	10	10	11	9	9	11	46	45	52	55	91	11	11	12	x		
case13659pegaselimit	8	9	8	8	9	8	8	8	9	9	33	48	90	8	10	o	x		

Figure 23: LU factorizations per system and configuration in S1 (the paper's parameters). x: fails; o: converges to another root. Bold: cheaper than the paper's method or solves a system it does not.

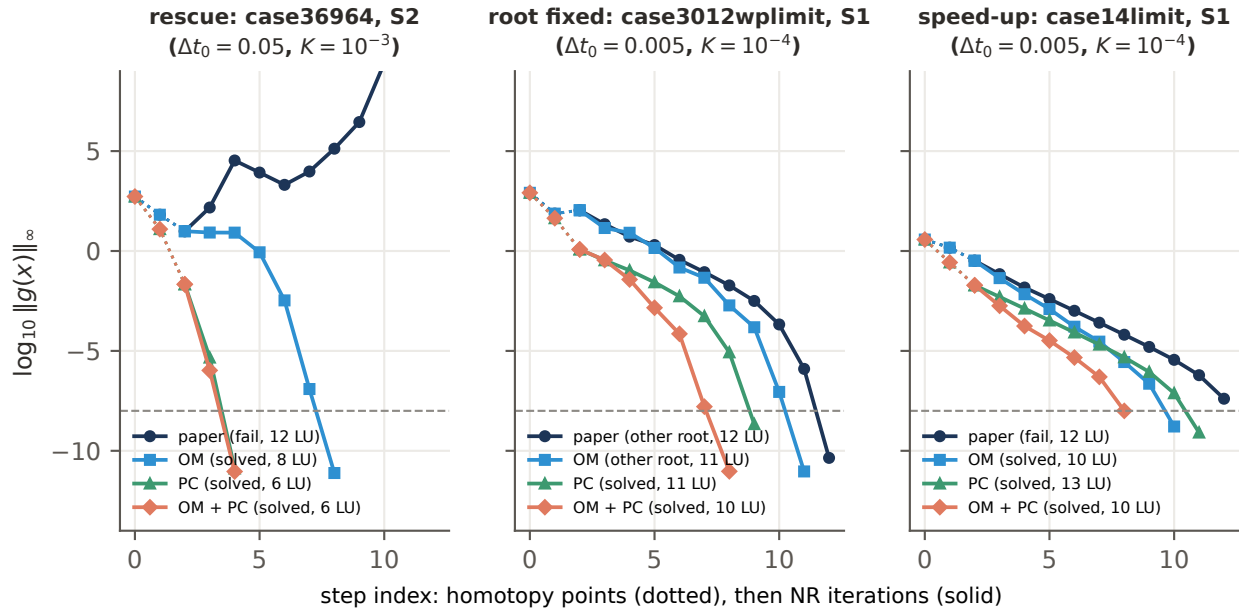


Figure 24: Mismatch along the path (dotted) and through NR (solid) for three runs the paper's method does not solve: a rescue (case36964, S2), a root fix (case3012wplimit) and a speed-up (case14limit). Dashed: tolerance 10^{-8} .

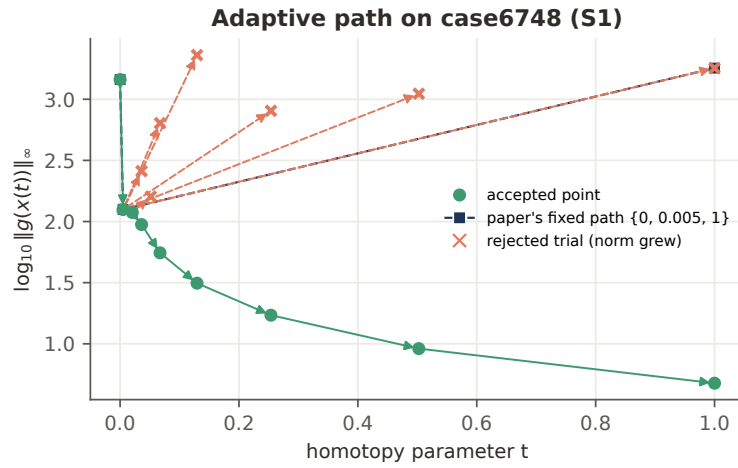


Figure 25: Every step the adaptive rule (with the multiplier) tries on case6748, S1. Green: accepted; orange: rejected trials, which still cost an LU. Navy squares: the paper's fixed path.

D Member Contributions

All five members contributed equally. Every member owns one part of the core code, one part of the paper’s results, one modification and one numerical study from the proposal, the report figures and tables built from that work, and the tests for their own code (Table 15). The details follow by roll number; the full account is in CONTRIBUTIONS.md in the repository [21].

Table 15: Contributions of each group member, by roll number.

Roll, member	Core code	Paper results	Modifications and studies	Report and tests
2105039 Mohiuddin Sizan	Table output in CSV and Markdown, paper values side by side, timing benchmark	Tables 2–6, Figure 3	FE, RK2, RK4 and BE-chord step rules; golden-section tuning of δ	Tables 2–5 in L ^A T _E X, fidelity plot, tuning figure; Table 4 regression test, explicit first step, golden section
2105040 Sadman bin Tareq	Sparse LU with the LU counter, result types, figure style	Section 4.4, Figure 5	Richardson step control, scaled homotopy; Gauss elimination and LU from scratch; command-line tool	Jacobian sparsity, Section 4.4 grid, Figure 5 cost, Richardson paths; tests of Gauss/LU, pivoting, Richardson, scaled homotopy, CLI
2105057 Mosharaf Hossain Apurbo	Fixed-point homotopy, FE/BE/RK2 integrators, path schedules, hybrid method	Figures 1, 2, 4, Section 4.2.1	solve() with switches, Newton corrector, adaptive steps; spectrum of the Jacobian	Tutorial geometry, first step, adaptive path, spectrum figures; tests of path, first step, tutorial, switch-off identity, corrector, eigenvalues
2105058 Elias Hasbi	NR, FDXB (XB), reconstruction of GSH-NR	Tables 7, 8	Optimal multiplier, Newton homotopy; NR and FDXB with our own LU	Timing, cost against robustness, per-case map, rescue histories; tests of NR/FDXB counts, multiplier, own-LU iterates, Newton homotopy
2105060 Jonayed Mohiuddin Shanto	Public GitHub repository; power flow model (g, J , case reader) and the solver interface; test-system download, experiment runner	Flat-start check and base-MVA finding, root check of Table 7, audit of the paper	Wider test bed of 34 systems, solution (root) check, the comparison study of all modifications; feasibility maps	Test-systems table and figure, audit table, feasibility figure, study summary figure and table; Jacobian test

2105039, Mohiuddin Sizan. Wrote the table output that prints every reproduced value next to the paper’s, and the median-of-runs timing benchmark. Reproduced the paper’s Tables 2–6 and Figure 3, including the paper’s one failure on case36964. Added the FE, RK2, RK4 and BE-chord step rules and showed, with Sadman’s Richardson control, that BE stays the cheapest integrator under error control (10.6 LUs in S1 against 37, 48 and 87). Built the golden-section tuning of δ and the test of fixed and spectral rules (Table 8).

2105040, Sadman bin Tareq. Wrote the sparse LU wrapper whose counter is the cost measure of the whole project, the result types and the figure style. Reproduced Section 4.4 and Figure 5. Built Richardson step control, the only change that solves some systems nothing else solves, and the scaled homotopy; wrote Gauss elimination and LU with partial pivoting from scratch, and the command-line tool `python -m improvements`.

2105057, Mosharaf Hossain Apurbo. Implemented the core of the paper: the fixed-point homotopy, its three integrators and the hybrid method, and showed that the BE first step equals the paper’s linearised static homotopy. Reproduced Figures 1, 2, 4 and Section 4.2.1. Built the `solve()` framework that switches every modification on and off, the Newton corrector (152 runs solved, none lost) and adaptive steps, and the spectral study of the flat-start Jacobian.

2105058, Elias Hasbi. Wrote NR, FDXB and the reconstruction of GSH-NR, and found the paper’s way of counting FDXB iterations. Reproduced Tables 7 and 8. Built the optimal multiplier, which with Apurbo’s corrector gives the recommended configuration (152 runs, 7% fewer LUs), the Newton homotopy, and NR and FDXB running on our own LU (Table 9).

2105060, Jonayed Mohiuddin Shanto. Set up and maintains the project’s public GitHub repository [21], where the code of all five members comes together. Built the power flow model that every other part runs on: the case reader, the equations $g(x)$ and the Jacobian $J(x)$, and the `g/jacobian` interface against which every solver, integrator and modification of the team is written, so that the same code runs on the 2×2 tutorial and on the 109,272-bus grid. Wrote the download of the test systems and the runner of all 15 experiments, which reproduces the whole project with one command. Found the base-MVA inconsistency of the two small limit cases, checked every Table 7 run against the reference solution, and compiled the audit of the paper (Table 6). Widened the test bed from 9 to 34 systems and wrote the solution (root) check behind the project’s main finding—14 of the paper’s 141 converged runs reach another root. That check is used throughout the project: in Table 7, the comparison study, the feasibility maps, the tuning of δ and the command-line tool. Wrote the comparison study in which the work of all members meets: it runs every member’s modification, alone and combined, on 34 systems under five settings (2,890 runs), and separates rescues, root fixes and speed-ups (Table 7). Built the feasibility maps of the $(K, \Delta t_0)$ plane, which explain the paper’s unexplained failure at $\Delta t_0 = 0.1$.